Sniper Bot -- Paper-Only Hardening and Android Setup Guide

Repository: https://github.com/drp-wq/Sniper | Runtime: Node.js >= 18 (Zero External NPM Dependencies) | Roles: AGY (Independent Auditor) & OpenCode (Primary Implementer)

Table of Contents

* 1. Project Overview and Paper-Only Safety Warning
* 2. Repository Directory Tree and File Reference
* 3. Approved Implementation Plan and Completed Checkpoints
* 4. Numbered Steps to Reproduce the Changes in the Repository
* 5. Exact GitHub / iPad Workflow (Adding, Reviewing, Committing, Pushing)
* 6. Exact Android / Termux Installation, Wake Lock, and Verification Commands
* 7. Cloudflare Worker Configuration and Deployment Steps
* 8. Required Environment Variables (Placeholders Only)
* 9. Test Commands and Expected Results (Actual Verified Output)
* 10. How to Start, Stop, Halt, and Inspect the Paper Bot
* 11. How to Generate and Interpret Reports (200-Trade Evaluation Gate)
* 12. Troubleshooting, Known Limitations, and Rollback Instructions
* 13. Tested vs. Unverified Matrix
* 14. Complete Final Contents of All Changed and Added Files

1. Project Overview and Paper-Only Safety Warning

SAFETY WARNING -- PAPER-TRADING ONLY: This system is strictly a Solana signal-scanning and paper-trading evaluation engine. It contains NO private keys, NO seed phrases, NO wallet secrets, NO transaction signing, and NO live order execution. Never paste private keys or real API secrets into chat, issues, commits, or public logs. Even if paper results meet the 200-trade evaluation gate ('Paper results approved'), that phrase only authorizes discussing a separate future phase and NEVER enables live trading in this codebase.

Architecture Flow: Helius Enhanced Webhook -> Cloudflare Worker (KV ring buffer, max 50 items) -> Android/Termux phone polls GET /signals -> On-chain SPL mint/authority/concentration gate (src/gate.js) -> Simulated paper entry with pessimistic entry slippage and pool-pair pinning (src/paper.js & src/price.js) -> Periodic exit evaluation (stop loss, liquidity drop, max hold, trailing stop, partial take profit, time stop) -> Structured JSONL logging (logs/paper.jsonl) and atomic disk position persistence (logs/positions.json) -> Statistical reporting (src/report.js).

2. Repository Directory Tree and File Reference

Repository Structure:

```
sniper-bot/
+-- .env.example          # Environment variable template with safe placeholders only
+-- .gitignore            # Excludes .env, logs/, node_modules/, HALT, *.key, keystore*
+-- package.json          # Zero-dependency ESM package definition (start, report, test)
+-- README.md             # Quick-start overview and paper-only commands
+-- run.sh                # Termux launcher script (wake-lock, git pull, tmux loop)
+-- docs/
|   '-- sniper-bot-paper-only-hardening-and-android-setup-guide.pdf
+-- src/
|   +-- api.js            # HTTP client for Worker (/signals, /notify, /heartbeat)
|   +-- config.js         # Zero-dependency .env loader and range-validated configuration
|   +-- gate.js           # On-chain SPL authority & owner-aggregated holder concentration check
|   +-- log.js            # Structured JSONL logger writing to stdout and logs/paper.jsonl
|   +-- main.js           # Sequential signal poller, queue-gap detector, and lifecycle runner
|   +-- paper.js          # Paper execution engine, atomic MAX_OPEN guard, disk state persistenc
    e
|   +-- price.js          # DexScreener quote fetcher with entry pool-pair preference
|   +-- report.js         # Statistical analyzer (exact median, drawdown, 2x slippage sensitivit
```

```
    y)
|   +-- rpc.js            # Solana JSON-RPC helper with 8-second timeout
|   ‘-- signals.js        # Pure helpers for signalKey, signalAgeMs, and sequenceGap
+-- test/
|   +-- gate.test.js      # Unit tests for authority checks and owner-wallet aggregation
|   +-- paper.test.js     # Unit tests for pool locking, missing quotes, partial TP, persistence
|   +-- report.test.js    # Unit tests for odd/even median, drawdown, legacy/new JSONL parsing
|   ‘-- signals.test.js   # Unit tests for signal identity, queue gaps, config bounds, MAX_OPEN
    race
‘-- worker/
    ‘-- index.js          # Cloudflare Worker relay (/hook, /signals, /heartbeat, /notify, /halt
    )
```

3. Approved Implementation Plan and Completed Checkpoints

* Baseline Commit (89b7673): Extracted unmodified sniper-bot.zip into the repository root so all hardening changes are auditable against the original code.
* Checkpoint 1 (a90941f) -- Signal Reliability, Atomic Entry Control & Config Validation: Added src/signals.js (deduplicating by sig:mint or queue:id:mint instead of permanently suppressing a mint); updated worker/index.js to return oldestId and deduplicate by (sig, mint); updated src/main.js to process signals sequentially and log signal_gap, stale_signal, and backward-compatible lagMs; hardened src/config.js with Number.isFinite and range validation; added in-flight opening Set and post-await MAX_OPEN check in src/paper.js.
* Checkpoint 2 (033dfa2) -- Realistic Paper Execution, Explicit Missing-Quote Handling & Position Persistence: Updated src/price.js to prefer the entry pairAddress on exit quotes and flag pairSwitched; removed fabricated -100% no_quote_total_loss from src/paper.js and replaced it with paper_unresolved (retaining last known price and liquidity valuation) plus paper_quote_recovered; added 2x slippage PnL (pnl2xSlip) calculation; implemented atomic disk persistence (logs/positions.json) and startup recovery.
* Checkpoint 3 (867ab7f) -- Honest Reporting, Drawdown, 2x Slippage Sensitivity & Gate Owner Aggregation: Fixed even-sample median bug in src/report.js; added cumulative pct-point and 10%-stake compounded equity max drawdown; added 2x slippage sensitivity comparison, unresolved position counts, malformed JSONL resilience, and 200-trade evaluation gate status; updated src/gate.js to aggregate multiple token accounts owned by the same wallet and handle unfunded system wallets.
* Checkpoint 4 -- RPC Status/Redaction, Unresolved Archival, Entry Quote Retries & Idempotent Shutdown: Hardened src/rpc.js and src/api.js with HTTP status checks before JSON parsing and secret/URL/header redaction; added MAX_UNRESOLVED_MIN (default 30m) archival in src/paper.js (emitting paper_archived_unresolved, verifying atomic disk persistence before freeing the MAX_OPEN slot, and never resurrecting archived positions or fabricating a paper_close PnL); added bounded entry quote retries (ENTRY_QUOTE_RETRIES=2, ENTRY_QUOTE_RETRY_MS=3000) with HALT checks before/after every wait; added idempotent SIGINT/SIGTERM/SIGHUP shutdown handler in src/main.js; updated src/report.js with archived unresolved counts, worst-case stress estimate, and >5% bias warning.

4. Numbered Steps to Reproduce the Changes in the Repository

* Step 1: Ensure Node.js >= 18 and Git are installed (‘node -v && git --version‘).
* Step 2: Clone or pull the repository (‘git clone https://github.com/drp-wq/Sniper.git && cd Sniper‘).
* Step 3: Verify the commit history includes baseline (89b7673) and Checkpoints 1-3 (a90941f, 033dfa2, 867ab7f), or copy the complete file listings from Section 14 of this PDF into their exact paths.
* Step 4: Run syntax verification across all source, worker, and test files (‘for f in src/*.js worker/*.js test/*.js; do node --check "$f"; done‘).
* Step 5: Run the built-in Node test suite (‘npm test‘ or ‘node --test‘). Confirm all 18 tests pass with 0 failures.

* Step 6: Verify the report script runs cleanly without requiring a '.env' file ('npm run report').

5. Exact GitHub / iPad Workflow (Adding, Reviewing, Committing, Pushing)

When operating from an iPad (via GitHub Mobile, Safari web editor, or GitHub Codespaces / Working Copy):

GitHub / Codespaces / Terminal Commands (iPad Friendly):

```
# 1. Inspect current branch, status, and recent commits
git status --short
git log -n 5 --oneline

# 2. Review exact diffs before staging or pushing
git diff 89b7673..HEAD --stat
git diff 89b7673..HEAD

# 3. Run full verification suite before pushing
npm test
npm run report

# 4. Stage, commit (if making adjustments), and push to origin/main
git push origin main
```

Tip for iPad Safari / GitHub Web UI: Press '.' (period) on a hardware keyboard or open the repository in GitHub Codespaces to review side-by-side diffs and run 'npm test' in the integrated terminal before pushing.

6. Exact Android / Termux Installation, Wake Lock, and Verification Commands

Install Termux and Termux:API (or Termux:Boot if auto-start is desired) from F-Droid (not outdated Play Store builds). Disable Android battery optimization / phantom process killer for Termux.

Android / Termux Setup & Verification Commands:

```
# 1. Update Termux packages and install Node.js LTS, Git, tmux, nano, and termux-api
pkg update && pkg upgrade -y
pkg install -y nodejs-lts git tmux nano termux-api

# 2. Clone the repository into Termux home directory
cd ~
git clone https://github.com/drp-wq/Sniper.git sniper-bot
cd ~/sniper-bot

# 3. Create your local .env from .env.example (NEVER commit .env)
cp .env.example .env
nano .env

# 4. Verify syntax and run the 18-test verification suite on the phone
for f in src/*.js worker/*.js test/*.js; do node --check "$f"; done
npm test
npm run report

# 5. Make run.sh executable and launch inside tmux with wake-lock
chmod +x run.sh
./run.sh
```

7. Cloudflare Worker Configuration and Deployment Steps

* 1. In the Cloudflare Dashboard -> Workers & Pages -> KV, create a KV namespace (e.g., 'SNIPER_KV').
* 2. Create or open your Worker (e.g., 'sniper-relay') -> Settings -> Bindings -> Add KV Namespace binding with Variable name EXACTLY 'KV' bound to your namespace.
* 3. Under Settings -> Variables and Secrets, add encrypted Secrets: 'APP_KEY' (long

random string matched in phone '.env'), 'HELIUS_AUTH' (exact Authorization header configured in Helius webhook), and optional 'TG_TOKEN' and 'TG_CHAT' for Telegram alerts.

* 4. Under Triggers -> Cron Triggers, add '* * * * *' (every minute) if you want the dead-man's heartbeat alert.
* 5. Paste the complete contents of 'worker/index.js' (Section 14) into the Worker Quick Edit editor and click Deploy.
* Free-Tier & Consistency Caveats: Cloudflare Workers Free tier has a 100,000 requests/day limit and KV Free tier has a 1,000 writes/day limit. High-volume Helius webhooks can exhaust KV free writes quickly. Furthermore, KV is eventually consistent: simultaneous webhooks can experience read-modify-write races. Reliable atomic queueing would require Cloudflare Durable Objects or Queues (paid plan).

8. Required Environment Variables (Placeholders Only)

File: .env.example (Copy to .env on Android device only)

```
# Copy to .env on the ANDROID device only. Never commit .env.
RPC_URL=https://mainnet.helius-rpc.com/?api-key=YOUR_HELIUS_KEY
WORKER_URL=https://sniper-relay.YOURNAME.workers.dev
APP_KEY=long-random-string-same-as-worker-secret
# Optional tuning
# POLL_MS=3000
# PRICE_MS=5000
# MAX_SIGNAL_AGE_MS=60000
# MAX_OPEN=5
# MIN_LIQ_USD=15000
# MAX_TOP10_PCT=35
# ENTRY_SLIP=0.05
# EXIT_SLIP=0.08
# ENTRY_QUOTE_RETRIES=2
# ENTRY_QUOTE_RETRY_MS=3000
# MAX_UNRESOLVED_MIN=30
```

9. Test Commands and Expected Results (Actual Verified Output)

Verified Command Output (Node v24.21.0):

```
$ for f in src/*.js worker/*.js test/*.js; do node --check "$f" && echo "SYNTAX OK: $f"; done &&
     npm test
SYNTAX OK: src/api.js
SYNTAX OK: src/config.js
SYNTAX OK: src/gate.js
SYNTAX OK: src/log.js
SYNTAX OK: src/main.js
SYNTAX OK: src/paper.js
SYNTAX OK: src/price.js
SYNTAX OK: src/report.js
SYNTAX OK: src/rpc.js
SYNTAX OK: src/signals.js
SYNTAX OK: worker/index.js
SYNTAX OK: test/gate.test.js
SYNTAX OK: test/paper.test.js
SYNTAX OK: test/report.test.js
SYNTAX OK: test/signals.test.js

> test
> node --test

[PASS] runGate rejects tokens when mintAuthority or freezeAuthority is still active
[PASS] runGate aggregates multiple token accounts owned by the same wallet and excludes PDA vaul
    ts
[PASS] rpc checks HTTP status (429/500), handles malformed JSON, and redacts API keys on network
    /RPC errors
[PASS] selectPair prefers entry pairAddress when still valid and flags fallback when switched
[PASS] missing quotes do not fabricate a -100% total loss, reset missingSince on recovery, and c
    an close cleanly
[PASS] long-unresolved positions archive safely after maxUnresolvedMin, never resurrect on resta
    rt, and keep slot on disk failure
```

```
[PASS] openPaper retries missing entry quotes up to entryQuoteRetries and aborts immediately if
    HALT activates
[PASS] partial take-profit + trailing stop and doubled-slippage PnL calculation are accurate
[PASS] positions persist to disk and recover safely on restart without duplication
[PASS] median calculates exact middle for odd and average of two middles for even arrays
[PASS] computeDrawdown tracks both cumulative pct-point drawdown and compounded 10%-stake equity
     drawdown
[PASS] analyzeLogs is backward-compatible with legacy JSONL records, tracks archived unresolved
    separately, and handles malformed lines safely
[PASS] signal identity distinguishes transactions for the same mint
[PASS] signal identity deduplicates the same transaction and mint
[PASS] unsigned signals use queue identity instead of mint-only identity
[PASS] signal age is measurable and missing timestamps are explicit
[PASS] queue gap counts IDs that could have fallen out of the bounded queue
[PASS] config numeric validation rejects NaN and out-of-range values
[PASS] openPaper enforces MAX_OPEN atomically across concurrent entry attempts
[PASS] pollOnce processes signals sequentially, logs queue gaps, skips stale signals, and preser
    ves lagMs
[PASS] worker deduplicates by (sig, mint) while allowing new transactions for the same mint
[PASS] shutdown handler is idempotent across SIGINT/SIGTERM/SIGHUP, persists state first, and do
    es not exit in tests
[INFO] tests 22 | suites 0 | pass 22 | fail 0
```

10. How to Start, Stop, Halt, and Inspect the Paper Bot

Operational Commands:

```
# Start or restart in background tmux session (acquires wake-lock, pulls latest fast-forward, ru
    ns loop)
./run.sh

# Attach to live tmux console (press Ctrl+B then D to detach without stopping)
tmux attach -t bot

# Stream structured JSONL logs in real time
tail -f logs/paper.jsonl

# Inspect currently persisted open paper positions
cat logs/positions.json

# Activate local kill switch (blocks all new entries immediately while continuing exit ticks for
     open positions)
touch HALT

# Resume new entries locally
rm -f HALT

# Activate or clear remote Worker kill switch from iPad/phone
curl -X POST -H "x-key: YOUR_APP_KEY" "https://sniper-relay.YOURNAME.workers.dev/halt?on=1"
curl -X POST -H "x-key: YOUR_APP_KEY" "https://sniper-relay.YOURNAME.workers.dev/halt?on=0"

# Stop the bot completely (saves open positions to logs/positions.json on SIGINT)
tmux kill-session -t bot
termux-wake-unlock || true
```

11. How to Generate and Interpret Reports (200-Trade Evaluation Gate)

Run ‘npm run report‘ (or ‘node src/report.js logs/paper.jsonl‘). The hardened report computes:

* Signals Gated, Passed, Gate Errors & Queue Gaps: Tracks how many signals passed ‘src/gate.js‘ and whether any signals fell out of the Worker’s 50-item KV buffer (‘signal_gap‘).
* Relay Lag (median & p90): Measures elapsed time from Worker webhook receipt (‘s.ts‘) to phone gate completion. Note: This measures Worker->phone relay + RPC gate latency, not on-chain block confirmation to Worker latency (unless compared against ‘chainTs‘).
* Win Rate, Exact Mean & Median PnL (1x and 2x Slippage): Uses exact even/odd median

calculation and compares baseline fills (‘ENTRY_SLIP=5%‘, ‘EXIT_SLIP=8%‘) against doubled slippage (‘10%‘ entry, ‘16%‘ exit).
* Maximum Drawdown: Reports both cumulative percentage-point drawdown and a documented 10%-per-trade compounded equity-curve maximum drawdown.
* Unresolved / Missing-Quote Tracking: Reports currently open positions and count of ‘paper_unresolved‘ events so missing DexScreener quotes are never hidden or falsified as ‘-100%‘ closed trades.
* 200-Trade Evaluation Gate: Requires ‘closedCount >= 200‘, ‘meanPnl > 0‘, ‘medianPnl > 0‘, ‘meanPnl2x > 0‘, and ‘medianPnl2x > 0‘. Even at 200 trades, check whether unresolved positions or high concentration in a single market regime bias the sample before drawing conclusions.

12. Troubleshooting, Known Limitations, and Rollback Instructions

* Gate Limitations (Not a Complete Rug Detector): ‘src/gate.js‘ verifies revoked ‘mintAuthority‘/‘freezeAuthority‘ and checks that the top-10 wallet owners among the 20 largest token accounts hold ‘<= MAX_TOP10_PCT‘. It CANNOT detect un-burned LP tokens, sybil wallets split across >20 accounts, malicious Token-2022 transfer hooks, or post-entry developer liquidity pulls.
* DexScreener Indexing & Price Impact: Newly created Solana pools take seconds to minutes to appear on DexScreener and quotes reflect spot price + reported USD liquidity, not full AMM constant-product depth for large order sizes.
* Stuck Unresolved Positions: If a token is permanently delisted from DexScreener, it stays in ‘logs/positions.json‘ with ‘unresolvedMissingQuote: true‘ rather than fabricating a ‘-100%‘ loss. To free its ‘MAX_OPEN‘ slot manually, stop the bot, archive the entry from ‘logs/positions.json‘, and restart.

Exact Rollback Commands:

```
# Rollback Checkpoint 3 only (return to Checkpoint 2):
git revert 867ab7f

# Rollback Checkpoint 2 and 3 (return to Checkpoint 1):
git revert 867ab7f 033dfa2

# Reset working tree completely to unmodified sniper-bot.zip baseline (commit 89b7673):
git checkout 89b7673 -- .
```

13. Tested vs. Unverified Matrix

* TESTED & VERIFIED IN REPOSITORY: Syntax checks (‘node --check‘) on all 15 JS files; 18 automated unit tests (‘npm test‘) covering signal keys, age/staleness, queue gaps, config validation, concurrent ‘MAX_OPEN‘ atomic locks, sequential polling, Worker ‘(sig, mint)‘ deduplication, entry pool-pair locking, missing-quote unresolved handling & recovery, partial TP + trailing stop + 2x slippage math, disk position persistence/recovery, authority + sybil wallet aggregation in ‘runGate‘, and odd/even median + drawdown + legacy/new JSONL parsing in ‘report.js‘.
* UNVERIFIED IN THIS SANDBOX (REQUIRES USER CREDENTIALS / LIVE DEVICE): Live Cloudflare Worker deployment and KV namespace binding; live Helius webhook delivery; live Solana RPC calls against Helius mainnet; live DexScreener API polling; physical Android Termux ‘termux-wake-lock‘ and ‘tmux‘ long-running session behavior.

14. Complete Final Contents of All Changed and Added Files

File Path: .env.example

```
# Copy to .env on the ANDROID device only. Never commit .env.
RPC_URL=https://mainnet.helius-rpc.com/?api-key=YOUR_HELIUS_KEY
WORKER_URL=https://sniper-relay.YOURNAME.workers.dev
APP_KEY=long-random-string-same-as-worker-secret
# Optional tuning
# POLL_MS=3000
# PRICE_MS=5000
# MAX_SIGNAL_AGE_MS=60000
# MAX_OPEN=5
```

```
# MIN_LIQ_USD=15000
# MAX_TOP10_PCT=35
# ENTRY_SLIP=0.05
# EXIT_SLIP=0.08
# ENTRY_QUOTE_RETRIES=2
# ENTRY_QUOTE_RETRY_MS=3000
# MAX_UNRESOLVED_MIN=30
```

File Path: package.json

```
{
  "name": "sniper-bot",
  "private": true,
  "type": "module",
  "engines": { "node": ">=18" },
  "scripts": {
    "start": "node src/main.js",
    "report": "node src/report.js",
    "test": "node --test"
  }
}
```

File Path: README.md

```
# sniper-bot (paper phase)

Zero-dependency Node (>=18) scanner + paper trader. No keys, no signing, no real trades.

Flow: Helius webhook -> Cloudflare Worker (KV) -> phone polls -> gate -> paper position -> exit
    rules -> logs/paper.jsonl

Run (Termux): ‘cp .env.example .env && nano .env && ./run.sh‘
Test suite: ‘npm test‘
Report: ‘npm run report‘ (or ‘node src/report.js‘)
Halt new entries: ‘touch HALT‘ (or POST /halt?on=1 on the Worker)
Setup guide: see ‘docs/sniper-bot-paper-only-hardening-and-android-setup-guide.pdf‘.
```

File Path: src/signals.js

```
// Pure helpers for signal identity and queue/latency checks.
// No dependencies; safe to test with Node’s built-in test runner.

export function signalKey(signal) {
  const mint = typeof signal?.mint === "string" ? signal.mint.trim() : "";
  const sig = typeof signal?.sig === "string" ? signal.sig.trim() : "";
  if (sig) return ‘${sig}:${mint}‘;
  // Without a transaction signature, deduplication cannot be reliable across retries.
  return ‘queue:${signal?.id ?? "unknown"}:${mint}‘;
}

export function signalAgeMs(signal, now = Date.now()) {
  const receivedAt = Number(signal?.ts);
  if (!Number.isFinite(receivedAt) || receivedAt <= 0) return null;
  return Math.max(0, now - receivedAt);
}

export function sequenceGap(since, oldestId) {
  if (!Number.isFinite(since) || since < 0 || !Number.isFinite(oldestId)) return 0;
  return Math.max(0, oldestId - since - 1);
}
```

File Path: src/config.js

```
import { readFileSync, existsSync } from "node:fs";

// Minimal .env loader (no dependencies).
export function parseEnvText(text, targetEnv = {}) {
  for (const line of String(text).split("\n")) {
    const m = line.match(/^\s*([A-Z0-9_]+)\s*=\s*(.*?)\s*$/);
    if (m && !(m[1] in targetEnv)) {
      targetEnv[m[1]] = m[2].replace(/^["’]|["’]$/g, "");
    }
  }
  return targetEnv;
```

```
}

if (existsSync(".env")) {
  parseEnvText(readFileSync(".env", "utf8"), process.env);
}

export function parseNum(env, key, defaultVal, { min = -Infinity, max = Infinity } = {}) {
  const raw = env[key];
  if (raw === undefined || raw === null || String(raw).trim() === "") return defaultVal;
  const val = Number(raw);
  if (!Number.isFinite(val) || val < min || val > max) {
    throw new Error(‘Invalid numeric env var ${key} (expected finite number in [${min}, ${max}])
    ‘);
  }
  return val;
}

export function loadConfig(env = process.env, { allowPlaceholder = false } = {}) {
  const need = (k, fallback = "") => {
    const v = env[k] !== undefined ? String(env[k]).trim() : "";
    if (!v) {
      if (allowPlaceholder) return fallback;
      throw new Error(‘Missing env var ${k} (see .env.example)‘);
    }
    return v;
  };

  return {
    rpcUrl: need("RPC_URL", "http://127.0.0.1:8899"),
    workerUrl: need("WORKER_URL", "http://127.0.0.1:8787").replace(/\/$/, ""),
    appKey: need("APP_KEY", "test-app-key"),
    pollMs: parseNum(env, "POLL_MS", 3000, { min: 250, max: 300000 }),
    priceMs: parseNum(env, "PRICE_MS", 5000, { min: 500, max: 300000 }),
    heartbeatMs: parseNum(env, "HEARTBEAT_MS", 5 * 60 * 1000, { min: 5000, max: 3600000 }),
    maxSignalAgeMs: parseNum(env, "MAX_SIGNAL_AGE_MS", 60000, { min: 1000, max: 3600000 }),
    maxOpen: Math.floor(parseNum(env, "MAX_OPEN", 5, { min: 1, max: 100 })),
    minLiqUsd: parseNum(env, "MIN_LIQ_USD", 15000, { min: 0, max: 1e9 }),
    // Pessimistic paper fills: you will not get the screen price.
    entrySlip: parseNum(env, "ENTRY_SLIP", 0.05, { min: 0, max: 0.95 }),
    exitSlip: parseNum(env, "EXIT_SLIP", 0.08, { min: 0, max: 0.95 }),
    entryQuoteRetries: Math.floor(parseNum(env, "ENTRY_QUOTE_RETRIES", 2, { min: 0, max: 5 })),
    entryQuoteRetryMs: parseNum(env, "ENTRY_QUOTE_RETRY_MS", 3000, { min: 100, max: 30000 }),
    gate: {
      maxTop10Pct: parseNum(env, "MAX_TOP10_PCT", 35, { min: 0.1, max: 100 }),
    },
    // Exit rules (percent unless noted). Tune these from the iPad.
    rules: {
      stopLoss: 25,      // exit at -25% from entry
      bandDown: 10,      // time-stop band: -10% ...
      bandUp: 15,        // ... to +15% of entry
      timeStopSec: 180,  // exit if sideways in band this long with no new high
      maxHoldMin: 30,    // exit remainder after 30 min
      takeProfit: 100,   // sell half at +100%
      trailArm: 50,      // arm trailing stop at +50%
      trail: 20,         // exit on 20% drawdown from peak once armed
      liqDrop: 30,       // exit if liquidity falls 30% from entry
      noQuoteTicks: 12,  // consecutive missing quotes before marking unresolved
      maxUnresolvedMin: parseNum(env, "MAX_UNRESOLVED_MIN", 30, { min: 1, max: 1440 }),
    },
  };
}

const isReportCli = Boolean(process.argv[1] && process.argv[1].endsWith("report.js"));
const isTestEnv =
  process.env.NODE_ENV === "test" ||
  process.env.NODE_TEST_CONTEXT !== undefined ||
  process.execArgv.some((arg) => arg.startsWith("--test")) ||
  isReportCli;
export const cfg = loadConfig(process.env, { allowPlaceholder: isTestEnv });
```

File Path: src/rpc.js

```
import { cfg } from "./config.js";

let id = 0;

export function redactSecrets(input, extraSecrets = []) {
  let out = String(input ?? "");
  const secrets = [
    cfg?.rpcUrl,
    cfg?.workerUrl,
    cfg?.appKey,
    ...extraSecrets,
  ].filter((s) => typeof s === "string" && s.trim().length > 0);

  // Strip full URLs with query strings or credentials first
  out = out.replace(/https?:\/\/[^\s"'<>]+/gi, (match) => {
    try {
      const u = new URL(match);
      return `${u.protocol}//${u.host}${u.pathname}${u.search ? "?[REDACTED]" : ""}`;
    } catch {
      return "[REDACTED_URL]";
    }
  });

  // Redact common query/header key patterns and Bearer tokens
  out = out.replace(/([?&](?:api[-_]?key|key|token|auth|secret)=)[^&\s"'<>]+/gi, "$1[REDACTED]")
    ;
  out = out.replace(/(authorization\s*[:=]\s*bearer\s+)[^\s,"'<>]+/gi, "$1[REDACTED]");
  out = out.replace(/(\bbearer\s+)[^\s,"'<>]+/gi, "$1[REDACTED]");
  out = out.replace(/((?:api[-_]?key|x-key|authorization)\s*[:=]\s*)[^\s,"'<>]+/gi, "$1[REDACTED
    ]");

  for (const s of secrets) {
    if (s.length >= 4) {
      out = out.split(s).join("[REDACTED]");
    }
    try {
      const u = new URL(s);
      for (const [, val] of u.searchParams.entries()) {
        if (val && val.length >= 4) {
          out = out.split(val).join("[REDACTED]");
        }
      }
    } catch {
      /* not a URL */
    }
  }

  return out.slice(0, 300);
}

export async function rpc(method, params, deps = {}) {
  const fetchFn = deps.fetch ?? fetch;
  const rpcUrl = deps.rpcUrl ?? cfg.rpcUrl;
  const extraSecrets = [rpcUrl, ...(deps.extraSecrets ?? [])];

  let r;
  try {
    r = await fetchFn(rpcUrl, {
      method: "POST",
      headers: { "content-type": "application/json" },
      body: JSON.stringify({ jsonrpc: "2.0", id: ++id, method, params }),
      signal: AbortSignal.timeout(8000),
    });
  } catch (e) {
    const rawMsg = e?.cause?.message ? `${e.message} (${e.cause.message})` : (e?.message || Stri
    ng(e));
    throw new Error(`${method}: network error: ${redactSecrets(rawMsg, extraSecrets)}`);
  }
```

```
  if (!r || typeof r.ok !== "boolean") {
    throw new Error(`${method}: invalid RPC response`);
  }

  if (!r.ok) {
    throw new Error(`${method}: HTTP ${r.status}`);
  }

  let j;
  try {
    j = await r.json();
  } catch {
    throw new Error(`${method}: malformed JSON response`);
  }

  if (!j || typeof j !== "object") {
    throw new Error(`${method}: invalid JSON-RPC payload`);
  }

  if (j.error) {
    const codePart = j.error.code !== undefined ? ` (code ${j.error.code})` : "";
    const safeMsg = redactSecrets(j.error.message ?? "RPC error", extraSecrets);
    throw new Error(`${method}${codePart}: ${safeMsg}`);
  }

  return j.result;
}
```

File Path: src/api.js

```
import { cfg } from "./config.js";
import { redactSecrets } from "./rpc.js";

const headers = { "x-key": cfg.appKey, "content-type": "application/json" };

export async function getSignals(since, deps = {}) {
  const fetchFn = deps.fetch ?? fetch;
  const workerUrl = deps.workerUrl ?? cfg.workerUrl;
  const extraSecrets = [workerUrl, cfg.appKey];

  let r;
  try {
    r = await fetchFn(`${workerUrl}/signals?since=${since}`, {
      headers,
      signal: AbortSignal.timeout(8000),
    });
  } catch (e) {
    const rawMsg = e?.cause?.message ? `${e.message} (${e.cause.message})` : (e?.message || Stri
    ng(e));
    throw new Error(`signals network error: ${redactSecrets(rawMsg, extraSecrets)}`);
  }

  if (!r.ok) throw new Error(`signals HTTP ${r.status}`);

  try {
    return await r.json();
  } catch {
    throw new Error("signals malformed JSON");
  }
}

export const notify = (text) =>
  fetch(`${cfg.workerUrl}/notify`, {
    method: "POST",
    headers,
    body: JSON.stringify({ text }),
    signal: AbortSignal.timeout(8000),
  }).catch(() => {});

export const heartbeat = (open) =>
  fetch(`${cfg.workerUrl}/heartbeat`, {
```

```
    method: "POST",
    headers,
    body: JSON.stringify({ open }),
    signal: AbortSignal.timeout(8000),
  }).catch(() => {});
```

File Path: src/price.js

```
// Free price + liquidity source for the paper phase (DexScreener).
// New pairs can take a short while to be indexed: a null result means "not yet".

export function selectPair(pairs, preferredPair = null) {
  const valid = (Array.isArray(pairs) ? pairs : []).filter(
    (p) =>
      p &&
      p.chainId === "solana" &&
      Number.isFinite(Number(p.priceUsd)) &&
      Number(p.priceUsd) > 0 &&
      Number.isFinite(Number(p.liquidity?.usd ?? 0)) &&
      Number(p.liquidity?.usd ?? 0) >= 0,
  );
  if (!valid.length) return null;

  valid.sort((a, b) => Number(b.liquidity?.usd ?? 0) - Number(a.liquidity?.usd ?? 0));

  let chosen = valid[0];
  let pairSwitched = false;
  if (preferredPair) {
    const match = valid.find((p) => p.pairAddress === preferredPair);
    if (match) {
      chosen = match;
    } else {
      pairSwitched = true;
    }
  }

  return {
    price: Number(chosen.priceUsd),
    liqUsd: Number(chosen.liquidity?.usd ?? 0),
    pair: chosen.pairAddress ?? null,
    pairSwitched,
  };
}

export async function getQuote(mint, opts = {}) {
  const preferredPair = typeof opts === "string" ? opts : (opts?.preferredPair ?? null);
  const fetchFn = opts?.fetch ?? fetch;
  try {
    const r = await fetchFn(`https://api.dexscreener.com/latest/dex/tokens/${mint}`, {
      signal: AbortSignal.timeout(8000),
    });
    if (!r.ok) return null;
    const j = await r.json();
    return selectPair(j?.pairs, preferredPair);
  } catch {
    return null;
  }
}
```

File Path: src/gate.js

```
import { rpc } from "./rpc.js";

const SYSTEM_PROGRAM = "11111111111111111111111111111111";

// Returns { pass, reasons[], top10Pct }. Any failed check rejects the token.
// Note: This gate only inspects on-chain mint/freeze authority and top-20 token account
// concentration. It CANNOT detect un-burned LP tokens, off-chain wallet bundling/sybils,
// honeypot transfer hooks, or post-entry liquidity withdrawals.
export async function runGate(mint, gateCfg, deps = {}) {
  const rpcFn = deps.rpc ?? rpc;
  const reasons = [];
```

```
  if (!mint || typeof mint !== "string") {
    return { pass: false, reasons: ["invalid mint address"], top10Pct: null };
  }

  // 1) Authorities must be revoked.
  const acc = await rpcFn("getAccountInfo", [mint, { encoding: "jsonParsed" }]);
  const info = acc?.value?.data?.parsed?.info;
  if (!info || !Number.isFinite(Number(info.decimals))) {
    return { pass: false, reasons: ["not a parsed SPL mint"], top10Pct: null };
  }
  if (info.mintAuthority) reasons.push("mint authority not revoked");
  if (info.freezeAuthority) reasons.push("freeze authority not revoked");

  // 2) Holder concentration, counting wallet-owned accounts and aggregating by owner wallet.
  //    Pool vaults / bonding curves are owned by programs (PDAs), so they are excluded.
  //    Unfunded system accounts (ownerInfo === null) still represent wallet addresses.
  const supply = Number((await rpcFn("getTokenSupply", [mint]))?.value?.uiAmount ?? 0);
  if (!Number.isFinite(supply) || supply <= 0) {
    reasons.push("invalid or zero token supply");
    return { pass: false, reasons, top10Pct: 100 };
  }

  const largest = ((await rpcFn("getTokenLargestAccounts", [mint]))?.value ?? []).slice(0, 20);
  const jp = { encoding: "jsonParsed" };
  const tokenAccs = largest.length
    ? (await rpcFn("getMultipleAccounts", [largest.map((a) => a.address), jp]))?.value ?? []
    : [];
  const owners = tokenAccs.map((a) => a?.data?.parsed?.info?.owner ?? null);
  const ownerKeys = [...new Set(owners.filter(Boolean))];
  const noData = { encoding: "base64", dataSlice: { offset: 0, length: 0 } };
  const ownerInfo = ownerKeys.length
    ? (await rpcFn("getMultipleAccounts", [ownerKeys, noData]))?.value ?? []
    : [];
  const isWallet = new Map(
    ownerKeys.map((k, i) => {
      const acct = ownerInfo[i];
      // A null account info means 0-lamport unfunded system address (still a wallet, not a depl
    oyed program PDA).
      return [k, acct === null || acct === undefined || acct?.owner === SYSTEM_PROGRAM];
    }),
  );

  // Aggregate balances by owner wallet so splitting across multiple token accounts cannot bypas
    s top-10 concentration.
  const byOwner = new Map();
  for (let i = 0; i < largest.length; i++) {
    const owner = owners[i];
    if (!owner || isWallet.get(owner) !== true) continue;
    const amt = Number(largest[i]?.uiAmount ?? 0);
    if (Number.isFinite(amt) && amt > 0) {
      byOwner.set(owner, (byOwner.get(owner) ?? 0) + amt);
    }
  }

  const top10WalletAmounts = [...byOwner.values()]
    .sort((a, b) => b - a)
    .slice(0, 10);

  const top10Pct = (top10WalletAmounts.reduce((s, amt) => s + amt, 0) / supply) * 100;
  if (top10Pct > gateCfg.maxTop10Pct) {
    reasons.push(`top10 wallets hold ${top10Pct.toFixed(1)}% (> ${gateCfg.maxTop10Pct}%)`);
  }

  return { pass: reasons.length === 0, reasons, top10Pct: Number(top10Pct.toFixed(1)) };
}
```

File Path: src/paper.js

```
import { existsSync, mkdirSync, readFileSync, renameSync, writeFileSync } from "node:fs";
import { dirname } from "node:path";
```

```
import { cfg } from "./config.js";
import { getQuote } from "./price.js";
import { log } from "./log.js";
import { notify } from "./api.js";

const R = cfg.rules;
const positions = new Map(); // mint -> position
const opening = new Set();   // mints currently awaiting entry quote
const archivedUnresolved = []; // archived positions after prolonged missing quotes
const MAX_ARCHIVED_HISTORY = 500;
const DEFAULT_STATE_FILE = "logs/positions.json";

export const openCount = () => positions.size;
export const isOpen = (mint) => positions.has(mint) || opening.has(mint);
export const getOpenPositions = () => [...positions.values()].map((p) => ({ ...p }));
export const getArchivedPositions = () => archivedUnresolved.map((p) => ({ ...p }));

export function savePositions(stateFile = DEFAULT_STATE_FILE, deps = {}) {
  if (!stateFile) return true;
  const logFn = deps.log ?? log;
  const writeFn = deps.writeFileSync ?? writeFileSync;
  const renameFn = deps.renameSync ?? renameSync;
  const mkdirFn = deps.mkdirSync ?? mkdirSync;

  try {
    mkdirFn(dirname(stateFile), { recursive: true });
    const tmpFile = `${stateFile}.tmp`;
    const payload = JSON.stringify(
      {
        updatedAt: new Date().toISOString(),
        positions: [...positions.values()],
        archivedUnresolved: archivedUnresolved.slice(-MAX_ARCHIVED_HISTORY),
      },
      null,
      2,
    );
    writeFn(tmpFile, payload, "utf8");
    renameFn(tmpFile, stateFile);
    return true;
  } catch (e) {
    logFn("error", { where: "savePositions", msg: String(e) });
    return false;
  }
}

export function loadPositions(stateFile = DEFAULT_STATE_FILE, deps = {}) {
  const logFn = deps.log ?? log;
  const maxOpen = deps.maxOpen ?? cfg.maxOpen;
  if (!stateFile || !existsSync(stateFile)) return 0;

  try {
    const raw = JSON.parse(readFileSync(stateFile, "utf8"));
    const list = Array.isArray(raw) ? raw : (Array.isArray(raw?.positions) ? raw.positions : [])
    ;
    const archivedList = Array.isArray(raw?.archivedUnresolved) ? raw.archivedUnresolved : [];

    archivedUnresolved.length = 0;
    const archivedMints = new Set();
    for (const arch of archivedList.slice(-MAX_ARCHIVED_HISTORY)) {
      if (arch && typeof arch.mint === "string" && arch.mint.trim()) {
        const cleanMint = arch.mint.trim();
        archivedUnresolved.push({ ...arch, mint: cleanMint });
        archivedMints.add(cleanMint);
      }
    }

    let restored = 0;
    for (const item of list) {
      if (positions.size >= maxOpen) break;
      if (
```

```
        !item ||
        typeof item.mint !== "string" ||
        !item.mint.trim() ||
        !Number.isFinite(Number(item.entry)) ||
        Number(item.entry) <= 0 ||
        !Number.isFinite(Number(item.openedAt))
      ) {
        continue;
      }
      const cleanMint = item.mint.trim();
      // Never resurrect archived positions on restart
      if (positions.has(cleanMint) || archivedMints.has(cleanMint)) continue;

      const entry = Number(item.entry);
      const restoredPos = {
        ...item,
        mint: cleanMint,
        sig: item.sig ?? null,
        entry,
        rawEntryQuote: Number.isFinite(Number(item.rawEntryQuote)) ? Number(item.rawEntryQuote)
    : null,
        entrySlip: Number.isFinite(Number(item.entrySlip)) ? Number(item.entrySlip) : cfg.entryS
    lip,
        peak: Number.isFinite(Number(item.peak)) && Number(item.peak) > 0 ? Number(item.peak) :
    entry,
        openedAt: Number(item.openedAt),
        liq0: Number.isFinite(Number(item.liq0)) ? Number(item.liq0) : 0,
        pair: item.pair ?? null,
        sidewaysSince: Number.isFinite(Number(item.sidewaysSince)) ? Number(item.sidewaysSince)
    : null,
        trailArmed: Boolean(item.trailArmed),
        tookHalf: Boolean(item.tookHalf),
        halfRawPrice: Number.isFinite(Number(item.halfRawPrice)) ? Number(item.halfRawPrice) : n
    ull,
        realized: Number.isFinite(Number(item.realized)) ? Number(item.realized) : 0,
        misses: Number.isFinite(Number(item.misses)) ? Math.max(0, Math.floor(Number(item.misses
    ))) : 0,
        missingSince: Number.isFinite(Number(item.missingSince)) ? Number(item.missingSince) : n
    ull,
        unresolvedMissingQuote: Boolean(item.unresolvedMissingQuote),
        lastQuotePrice: Number.isFinite(Number(item.lastQuotePrice)) ? Number(item.lastQuotePric
    e) : null,
        lastExecPrice: Number.isFinite(Number(item.lastExecPrice)) ? Number(item.lastExecPrice)
    : entry,
        lastLiqUsd: Number.isFinite(Number(item.lastLiqUsd)) ? Number(item.lastLiqUsd) : Number(
    item.liq0 ?? 0),
        lastQuoteAt: Number.isFinite(Number(item.lastQuoteAt)) ? Number(item.lastQuoteAt) : Numb
    er(item.openedAt),
      };
      positions.set(restoredPos.mint, restoredPos);
      restored += 1;
    }

    if (restored > 0 || archivedUnresolved.length > 0) {
      logFn("positions_restored", {
        count: restored,
        archivedUnresolvedCount: archivedUnresolved.length,
        mints: [...positions.keys()],
      });
    }
    return restored;
  } catch (e) {
    logFn("error", { where: "loadPositions", msg: String(e) });
    return 0;
  }
}

// Internal helper exposed for deterministic unit tests only.
export function _resetPaperStateForTests() {
  positions.clear();
```

```
  opening.clear();
  archivedUnresolved.length = 0;
}

const defaultSleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

export async function openPaper(mint, sig, meta = {}, deps = {}) {
  const quoteFn = deps.getQuote ?? getQuote;
  const logFn = deps.log ?? log;
  const notifyFn = deps.notify ?? notify;
  const maxOpen = deps.maxOpen ?? cfg.maxOpen;
  const minLiqUsd = deps.minLiqUsd ?? cfg.minLiqUsd;
  const entrySlip = deps.entrySlip ?? cfg.entrySlip;
  const entryQuoteRetries = deps.entryQuoteRetries ?? cfg.entryQuoteRetries;
  const entryQuoteRetryMs = deps.entryQuoteRetryMs ?? cfg.entryQuoteRetryMs;
  const haltCheck = deps.isHalted ?? (() => existsSync("HALT"));
  const sleepFn = deps.sleep ?? defaultSleep;
  const stateFile = deps.stateFile !== undefined ? deps.stateFile : DEFAULT_STATE_FILE;
  const nowFn = deps.now ?? Date.now;

  if (!mint || typeof mint !== "string") {
    return logFn("skip_invalid_mint", { mint: String(mint ?? ""), ...meta });
  }
  if (haltCheck()) {
    return logFn("halted_skip", { mint, reason: "local_halt_at_entry", ...meta });
  }
  if (positions.has(mint) || opening.has(mint)) {
    return logFn("signal_skip", { mint, reason: "position_already_open", ...meta });
  }
  if (positions.size + opening.size >= maxOpen) {
    return logFn("skip_maxopen", { mint, ...meta });
  }

  opening.add(mint);
  try {
    const maxAttempts = Math.max(1, 1 + Math.min(5, Math.max(0, Math.floor(entryQuoteRetries))))
    ;
    let q = null;
    let quoteAttempts = 0;

    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
      if (haltCheck()) {
        return logFn("halted_skip", { mint, reason: "halt_during_quote_retry", quoteAttempts, ..
    .meta });
      }
      quoteAttempts = attempt;
      q = await quoteFn(mint);
      if (q && Number.isFinite(Number(q.price)) && Number(q.price) > 0) {
        break;
      }
      q = null;
      if (attempt < maxAttempts) {
        if (haltCheck()) {
          return logFn("halted_skip", { mint, reason: "halt_before_quote_retry_wait", quoteAttem
    pts, ...meta });
        }
        await sleepFn(entryQuoteRetryMs);
        if (haltCheck()) {
          return logFn("halted_skip", { mint, reason: "halt_after_quote_retry_wait", quoteAttemp
    ts, ...meta });
        }
      }
    }

    if (!q) {
      return logFn("skip_noquote", { mint, quoteAttempts, ...meta });
    }
    if (haltCheck()) {
      return logFn("halted_skip", { mint, reason: "halt_before_paper_open", quoteAttempts, ...me
    ta });
```

```
    }
    if (q.liqUsd < minLiqUsd) {
      return logFn("skip_liquidity", { mint, liqUsd: q.liqUsd, quoteAttempts, ...meta });
    }
    // Atomic post-await re-check in case another entry completed concurrently.
    if (positions.has(mint)) {
      return logFn("signal_skip", { mint, reason: "position_already_open", ...meta });
    }
    if (positions.size >= maxOpen) {
      return logFn("skip_maxopen", { mint, ...meta });
    }

    const now = nowFn();
    const entry = q.price * (1 + entrySlip);
    positions.set(mint, {
      mint,
      sig: sig || null,
      entry,
      rawEntryQuote: q.price,
      entrySlip,
      peak: entry,
      openedAt: now,
      liq0: q.liqUsd,
      pair: q.pair ?? null,
      sidewaysSince: null,
      trailArmed: false,
      tookHalf: false,
      halfRawPrice: null,
      realized: 0,       // pct points already banked (weighted)
      misses: 0,
      missingSince: null,
      unresolvedMissingQuote: false,
      lastQuotePrice: q.price,
      lastExecPrice: entry,
      lastLiqUsd: q.liqUsd,
      lastQuoteAt: now,
      quoteAttempts,
      ...meta,
    });
    savePositions(stateFile, deps);
    logFn("paper_open", {
      mint,
      sig: sig || null,
      entry,
      rawEntryQuote: q.price,
      entrySlip,
      liqUsd: q.liqUsd,
      pair: q.pair ?? null,
      quoteAttempts,
      ...meta,
    });
    notifyFn(`PAPER BUY ${mint}\nentry ${entry}\nliq $${Math.round(q.liqUsd)}`);
  } finally {
    opening.delete(mint);
  }
}

// Pure rule engine. Returns null, { partial }, or { exit } with reason and pct.
export function evaluate(p, q, now = Date.now(), exitSlip = cfg.exitSlip) {
  const px = q.price * (1 - exitSlip);          // pessimistic executable price
  const pct = (px / p.entry - 1) * 100;
  const newHigh = px > p.peak;
  if (newHigh) p.peak = px;

  if (pct <= -R.stopLoss) return { exit: "stop_loss", pct, px };
  if (q.liqUsd < p.liq0 * (1 - R.liqDrop / 100)) return { exit: "liquidity_drop", pct, px };
  if (now - p.openedAt >= R.maxHoldMin * 60000) return { exit: "max_hold", pct, px };

  if (pct >= R.trailArm) p.trailArmed = true;
  if (p.trailArmed && px <= p.peak * (1 - R.trail / 100)) return { exit: "trailing_stop", pct, p
```

```
    x };

  if (!p.tookHalf && pct >= R.takeProfit) return { partial: "take_profit_half", pct, px };

  const inBand = pct >= -R.bandDown && pct <= R.bandUp;
  if (inBand && !newHigh) {
    p.sidewaysSince ??= now;
    if (now - p.sidewaysSince >= R.timeStopSec * 1000) return { exit: "time_stop", pct, px };
  } else {
    p.sidewaysSince = null;
  }
  return null;
}

export function computeDoubleSlipPnl({
  rawEntryQuote,
  rawExitQuote,
  halfRawPrice = null,
  tookHalf = false,
  entrySlip = cfg.entrySlip,
  exitSlip = cfg.exitSlip,
}) {
  if (!Number.isFinite(rawEntryQuote) || rawEntryQuote <= 0 || !Number.isFinite(rawExitQuote) ||
     rawExitQuote <= 0) {
    return null;
  }
  const entry2x = rawEntryQuote * (1 + Math.min(0.99, entrySlip * 2));
  const exitFactor2x = Math.max(0.01, 1 - exitSlip * 2);
  const exitPx2x = rawExitQuote * exitFactor2x;
  const finalPct2x = (exitPx2x / entry2x - 1) * 100;

  if (tookHalf && Number.isFinite(halfRawPrice) && halfRawPrice > 0) {
    const halfPx2x = halfRawPrice * exitFactor2x;
    const halfPct2x = (halfPx2x / entry2x - 1) * 100;
    return Number((0.5 * halfPct2x + 0.5 * finalPct2x).toFixed(2));
  }
  return Number(finalPct2x.toFixed(2));
}

function close(p, reason, pct, q, deps = {}) {
  const logFn = deps.log ?? log;
  const notifyFn = deps.notify ?? notify;
  const stateFile = deps.stateFile !== undefined ? deps.stateFile : DEFAULT_STATE_FILE;
  const nowFn = deps.now ?? Date.now;
  const exitSlip = deps.exitSlip ?? cfg.exitSlip;

  const weight = p.tookHalf ? 0.5 : 1;
  const pnl = p.realized + weight * pct;
  const pnl2xSlip = computeDoubleSlipPnl({
    rawEntryQuote: p.rawEntryQuote,
    rawExitQuote: q?.price,
    halfRawPrice: p.halfRawPrice,
    tookHalf: p.tookHalf,
    entrySlip: p.entrySlip ?? cfg.entrySlip,
    exitSlip,
  });

  positions.delete(p.mint);
  savePositions(stateFile, deps);

  const heldSec = Math.round((nowFn() - p.openedAt) / 1000);
  logFn("paper_close", {
    mint: p.mint,
    sig: p.sig ?? null,
    signalId: p.signalId ?? null,
    reason,
    pnl: Number(pnl.toFixed(2)),
    ...(pnl2xSlip !== null ? { pnl2xSlip } : {}),
    heldSec,
    entry: p.entry,
```

```
    exitPrice: q ? Number((q.price * (1 - exitSlip)).toFixed(12)) : null,
    rawEntryQuote: p.rawEntryQuote ?? null,
    rawExitQuote: q?.price ?? null,
    liqUsd: q?.liqUsd ?? null,
    pair: q?.pair ?? p.pair ?? null,
  });
  notifyFn(‘PAPER EXIT ${p.mint}\n${reason}  pnl ${pnl.toFixed(1)}%  held ${heldSec}s‘);
}

function archiveUnresolvedPosition(p, now, deps = {}) {
  const logFn = deps.log ?? log;
  const notifyFn = deps.notify ?? notify;
  const stateFile = deps.stateFile !== undefined ? deps.stateFile : DEFAULT_STATE_FILE;

  const lastUnrealizedPct =
    Number.isFinite(p.lastExecPrice) && p.entry > 0
      ? Number((((p.lastExecPrice / p.entry) - 1) * 100).toFixed(2))
      : null;
  const missingDurationSec = Number.isFinite(p.missingSince)
    ? Math.max(0, Math.round((now - p.missingSince) / 1000))
    : null;

  const archivedEntry = {
    ...p,
    archivedAt: now,
    missingDurationSec,
    lastUnrealizedPct,
  };

  // Stage removal and archival, then verify atomic persistence before committing
  positions.delete(p.mint);
  archivedUnresolved.push(archivedEntry);
  if (archivedUnresolved.length > MAX_ARCHIVED_HISTORY) {
    archivedUnresolved.shift();
  }

  const persisted = savePositions(stateFile, deps);
  if (!persisted) {
    // Rollback in-memory archival if disk persistence failed so we never silently lose state
    archivedUnresolved.pop();
    positions.set(p.mint, p);
    logFn("error", {
      where: "archiveUnresolvedPosition",
      mint: p.mint,
      msg: "failed to persist archived unresolved position; keeping active slot reserved",
    });
    return false;
  }

  logFn("paper_archived_unresolved", {
    mint: p.mint,
    sig: p.sig ?? null,
    signalId: p.signalId ?? null,
    reason: "missing_quote_timeout",
    misses: p.misses,
    missingDurationSec,
    entry: p.entry,
    tookHalf: Boolean(p.tookHalf),
    realizedBankedPct: Number((p.realized ?? 0).toFixed(2)),
    lastQuotePrice: p.lastQuotePrice ?? null,
    lastExecPrice: p.lastExecPrice ?? null,
    lastLiqUsd: p.lastLiqUsd ?? null,
    lastQuoteAt: p.lastQuoteAt ?? null,
    lastUnrealizedPct,
  });
  notifyFn(
    ‘PAPER ARCHIVED UNRESOLVED ${p.mint}\nmissing quotes for ${missingDurationSec ?? "?"}s; arch
    ived without synthetic close‘,
  );
  return true;
```

```
}

let busy = false;
export async function tick(deps = {}) {
  if (busy) return;
  busy = true;

  const quoteFn = deps.getQuote ?? getQuote;
  const logFn = deps.log ?? log;
  const notifyFn = deps.notify ?? notify;
  const stateFile = deps.stateFile !== undefined ? deps.stateFile : DEFAULT_STATE_FILE;
  const nowFn = deps.now ?? Date.now;
  const exitSlip = deps.exitSlip ?? cfg.exitSlip;
  const noQuoteTicks = deps.noQuoteTicks ?? R.noQuoteTicks;
  const maxUnresolvedMin = deps.maxUnresolvedMin ?? R.maxUnresolvedMin;

  try {
    let dirty = false;
    for (const p of [...positions.values()]) {
      const q = await quoteFn(p.mint, { preferredPair: p.pair });
      const now = nowFn();

      if (!q || !Number.isFinite(Number(q.price)) || Number(q.price) <= 0) {
        p.misses = (p.misses ?? 0) + 1;
        p.missingSince ??= now;
        dirty = true;
        // Missing quote is NOT a confirmed -100% fill. Mark as unresolved and retain last known
     valuation.
        if (p.misses >= noQuoteTicks && !p.unresolvedMissingQuote) {
          p.unresolvedMissingQuote = true;
          const lastUnrealizedPct =
            Number.isFinite(p.lastExecPrice) && p.entry > 0
              ? Number((((p.lastExecPrice / p.entry) - 1) * 100).toFixed(2))
              : null;
          logFn("paper_unresolved", {
            mint: p.mint,
            sig: p.sig ?? null,
            signalId: p.signalId ?? null,
            reason: "missing_quote_threshold",
            misses: p.misses,
            missingSince: p.missingSince,
            lastQuotePrice: p.lastQuotePrice ?? null,
            lastExecPrice: p.lastExecPrice ?? null,
            lastLiqUsd: p.lastLiqUsd ?? null,
            lastQuoteAt: p.lastQuoteAt ?? null,
            lastUnrealizedPct,
          });
          notifyFn(
            `PAPER UNRESOLVED ${p.mint}\nmissing ${p.misses} quotes; retaining last valuation (n
    ot closing at -100%)`,
          );
        }

        // Archive long-unresolved positions once continuous missing duration reaches maxUnresol
    vedMin
        if (
          p.unresolvedMissingQuote &&
          Number.isFinite(p.missingSince) &&
          now - p.missingSince >= maxUnresolvedMin * 60000
        ) {
          const archivedOk = archiveUnresolvedPosition(p, now, {
            ...deps,
            log: logFn,
            notify: notifyFn,
            stateFile,
          });
          if (archivedOk) {
            dirty = false; // archiveUnresolvedPosition already persisted state
          }
        }
```

```
        continue;
      }

      if (p.unresolvedMissingQuote) {
        logFn("paper_quote_recovered", {
          mint: p.mint,
          sig: p.sig ?? null,
          signalId: p.signalId ?? null,
          missesBeforeRecovery: p.misses,
          missingDurationSec: Number.isFinite(p.missingSince)
            ? Math.max(0, Math.round((now - p.missingSince) / 1000))
            : null,
          price: q.price,
          liqUsd: q.liqUsd,
        });
      }

      if (q.pairSwitched && p.pair && q.pair && q.pair !== p.pair) {
        logFn("paper_pair_switched", {
          mint: p.mint,
          previousPair: p.pair,
          newPair: q.pair,
          liqUsd: q.liqUsd,
        });
        p.pair = q.pair;
      }

      p.misses = 0;
      p.missingSince = null;
      p.unresolvedMissingQuote = false;
      p.lastQuotePrice = q.price;
      p.lastExecPrice = q.price * (1 - exitSlip);
      p.lastLiqUsd = q.liqUsd;
      p.lastQuoteAt = now;
      dirty = true;

      const res = evaluate(p, q, now, exitSlip);
      if (!res) continue;

      if (res.partial) {
        p.tookHalf = true;
        p.halfRawPrice = q.price;
        p.realized += 0.5 * res.pct;
        logFn("paper_partial", {
          mint: p.mint,
          sig: p.sig ?? null,
          signalId: p.signalId ?? null,
          reason: res.partial,
          pct: Number(res.pct.toFixed(1)),
          rawQuote: q.price,
          liqUsd: q.liqUsd,
        });
      } else {
        close(p, res.exit, res.pct, q, { ...deps, log: logFn, notify: notifyFn, stateFile, now:
    nowFn, exitSlip });
        dirty = false; // close() already persisted positions
      }
    }

    if (dirty) {
      savePositions(stateFile, deps);
    }
  } finally {
    busy = false;
  }
}
```

File Path: src/main.js

```
import { existsSync } from "node:fs";
import { fileURLToPath } from "node:url";
```

```
import { cfg } from "./config.js";
import { getSignals, heartbeat, notify } from "./api.js";
import { runGate } from "./gate.js";
import { openPaper, tick, openCount, isOpen, loadPositions, savePositions } from "./paper.js";
import { log } from "./log.js";
import { signalAgeMs, signalKey, sequenceGap } from "./signals.js";

// Inspect retained backlog at startup. Old signals are logged, not traded.
let since = 0;
const seen = new Set();
const SEEN_MAX = 5000;
let stopping = false;

function rememberKey(key) {
  seen.add(key);
  if (seen.size > SEEN_MAX) {
    const oldest = seen.values().next().value;
    if (oldest !== undefined) seen.delete(oldest);
  }
}

export async function handle(s, deps = {}) {
  const gateFn = deps.runGate ?? runGate;
  const openFn = deps.openPaper ?? openPaper;
  const isOpenFn = deps.isOpen ?? isOpen;
  const logFn = deps.log ?? log;
  const haltCheck = deps.isHalted ?? (() => existsSync("HALT"));
  const maxSignalAgeMs = deps.maxSignalAgeMs ?? cfg.maxSignalAgeMs;
  const nowFn = deps.now ?? Date.now;

  if (!s || typeof s.mint !== "string" || !s.mint.trim()) {
    logFn("signal_skip", {
      signalId: s?.id ?? null,
      reason: "invalid_signal_payload",
    });
    return;
  }

  const key = signalKey(s);
  const receivedAgeMs = signalAgeMs(s, nowFn());

  logFn("signal_received", {
    signalId: s.id,
    signalKey: key,
    mint: s.mint,
    sig: s.sig || null,
    receivedAgeMs,
  });

  if (receivedAgeMs === null) {
    logFn("signal_skip", {
      signalId: s.id,
      signalKey: key,
      mint: s.mint,
      reason: "missing_relay_timestamp",
    });
    return;
  }

  if (receivedAgeMs > maxSignalAgeMs) {
    logFn("signal_skip", {
      signalId: s.id,
      signalKey: key,
      mint: s.mint,
      reason: "stale_signal",
      receivedAgeMs,
      maxSignalAgeMs,
    });
    return;
  }
```

```
    if (haltCheck()) {
      logFn("halted_skip", {
        signalId: s.id,
        signalKey: key,
        mint: s.mint,
        reason: "local_halt",
      });
      return;
    }

    if (isOpenFn(s.mint)) {
      logFn("signal_skip", {
        signalId: s.id,
        signalKey: key,
        mint: s.mint,
        reason: "position_already_open",
      });
      return;
    }

    const gateStartedAt = nowFn();
    let g;
    try {
      g = await gateFn(s.mint, cfg.gate);
    } catch (e) {
      logFn("gate_error", {
        signalId: s.id,
        signalKey: key,
        mint: s.mint,
        sig: s.sig || null,
        receivedAgeMs,
        gateMs: nowFn() - gateStartedAt,
        msg: String(e),
      });
      return;
    }

    const gateMs = nowFn() - gateStartedAt;
    const endToGateMs = signalAgeMs(s, nowFn());

    // Keep ‘lagMs‘ for backward compatibility with src/report.js while adding explicit fields.
    logFn("gate", {
      signalId: s.id,
      signalKey: key,
      mint: s.mint,
      sig: s.sig || null,
      pass: g.pass,
      reasons: g.reasons,
      top10Pct: g.top10Pct,
      lagMs: receivedAgeMs,
      receivedAgeMs: endToGateMs,
      gateMs,
    });

    if (!g.pass) return;

    if (haltCheck()) {
      logFn("halted_skip", {
        signalId: s.id,
        signalKey: key,
        mint: s.mint,
        reason: "local_halt_after_gate",
      });
      return;
    }

    if (isOpenFn(s.mint)) {
      logFn("signal_skip", {
        signalId: s.id,
```

```
      signalKey: key,
      mint: s.mint,
      reason: "position_already_open",
    });
    return;
  }

  const entryStartedAt = nowFn();
  await openFn(s.mint, s.sig, {
    signalId: s.id,
    signalKey: key,
    receivedAgeMs: endToGateMs,
    gateMs,
  });

  logFn("entry_attempt_complete", {
    signalId: s.id,
    signalKey: key,
    mint: s.mint,
    entryAttemptMs: nowFn() - entryStartedAt,
    totalRelayToEntryMs: signalAgeMs(s, nowFn()),
  });
}

export async function pollOnce(deps = {}) {
  const getSignalsFn = deps.getSignals ?? getSignals;
  const handleFn = deps.handle ?? handle;
  const logFn = deps.log ?? log;
  const haltCheck = deps.isHalted ?? (() => existsSync("HALT"));

  const {
    signals = [],
    seq = since,
    oldestId = null,
    halt = false,
  } = await getSignalsFn(since);

  if (oldestId !== null) {
    const gap = sequenceGap(since, oldestId);
    if (gap > 0) {
      logFn("signal_gap", {
        since,
        oldestId,
        missingSequenceCount: gap,
      });
    }
  }

  if (Number.isFinite(seq) && seq >= since) {
    since = seq;
  }

  const halted = Boolean(halt) || haltCheck();

  for (const s of signals) {
    const key = signalKey(s);

    if (seen.has(key)) {
      logFn("signal_duplicate", {
        signalId: s?.id ?? null,
        signalKey: key,
        mint: s?.mint ?? null,
        sig: s?.sig || null,
      });
      continue;
    }

    rememberKey(key);

    if (halted || haltCheck()) {
```

```
      logFn("halted_skip", {
        signalId: s?.id ?? null,
        signalKey: key,
        mint: s?.mint ?? null,
        reason: "halt_active_at_poll",
      });
      continue;
    }

    // Sequential processing prevents concurrent gate checks from racing MAX_OPEN.
    try {
      await handleFn(s, deps);
    } catch (e) {
      logFn("error", {
        where: "handle",
        signalId: s?.id ?? null,
        signalKey: key,
        mint: s?.mint ?? null,
        msg: String(e),
      });
    }
  }
}

export function shutdown(signal = "SIGINT", deps = {}) {
  if (stopping) return false;
  stopping = true;

  const saveFn = deps.savePositions ?? savePositions;
  const logFn = deps.log ?? log;
  const openCountFn = deps.openCount ?? openCount;
  const isTest =
    process.env.NODE_ENV === "test" ||
    process.env.NODE_TEST_CONTEXT !== undefined ||
    process.execArgv.some((arg) => arg.startsWith("--test"));
  const exitFn = deps.exit ?? (isTest ? () => {} : (code) => process.exit(code));

  const persisted = Boolean(saveFn(deps.stateFile, deps));
  logFn("stop", {
    signal,
    persisted,
    openPositions: openCountFn(),
  });
  exitFn(0);
  return true;
}

export function _resetMainStateForTests(initialSince = 0) {
  since = initialSince;
  seen.clear();
  stopping = false;
}

async function pollLoop() {
  if (stopping) return;
  try {
    await pollOnce();
  } catch (e) {
    log("error", { where: "poll", msg: String(e) });
  }
  if (!stopping) {
    setTimeout(pollLoop, cfg.pollMs);
  }
}

export function startBot() {
  const restored = loadPositions();
  log("start", {
    pollMs: cfg.pollMs,
    paperOnly: true,
```

```
    startupBacklog: "inspect_retained_queue",
    maxSignalAgeMs: cfg.maxSignalAgeMs,
    restoredPositions: restored,
  });

  notify(`Bot started (paper mode, restored=${restored})`);
  pollLoop();

  setInterval(
    () => tick().catch((e) => log("error", { where: "tick", msg: String(e) })),
    cfg.priceMs,
  );

  setInterval(() => heartbeat(openCount()), cfg.heartbeatMs);
  heartbeat(openCount());

  for (const sig of ["SIGINT", "SIGTERM", "SIGHUP"]) {
    process.on(sig, () => shutdown(sig));
  }
}

const isDirectRun =
  process.argv[1] && fileURLToPath(import.meta.url) === process.argv[1];
if (isDirectRun) {
  startBot();
}
```

File Path: src/report.js

```
// Usage: node src/report.js   -> summary of paper results from logs/paper.jsonl
import { readFileSync, existsSync } from "node:fs";
import { fileURLToPath } from "node:url";
import { cfg } from "./config.js";
import { computeDoubleSlipPnl } from "./paper.js";

export const sum = (a) => a.reduce((s, x) => s + x, 0);
export const mean = (a) => (a.length ? sum(a) / a.length : 0);

// Exact median: averages the two middle elements for even-sized arrays.
export function median(a) {
  if (!a.length) return 0;
  const sorted = [...a].sort((x, y) => x - y);
  const mid = Math.floor(sorted.length / 2);
  return sorted.length % 2 === 1
    ? sorted[mid]
    : (sorted[mid - 1] + sorted[mid]) / 2;
}

export function percentile(a, p) {
  if (!a.length) return 0;
  const sorted = [...a].sort((x, y) => x - y);
  const idx = Math.min(sorted.length - 1, Math.max(0, Math.ceil((p / 100) * sorted.length) - 1))
    ;
  return sorted[idx];
}

// Equity-curve maximum drawdown using fixed-fractional stake per trade (default 10% of equity p
    er trade)
// and cumulative percentage-point drawdown (sum of trade PnL %).
export function computeDrawdown(pnlList, stakeFraction = 0.1) {
  let cumPct = 0;
  let peakCumPct = 0;
  let maxDdPctPoints = 0;

  let equity = 100;
  let peakEquity = 100;
  let maxDdEquityPct = 0;

  for (const rawPnl of pnlList) {
    const pnl = Number(rawPnl);
    if (!Number.isFinite(pnl)) continue;
```

```
    cumPct += pnl;
    if (cumPct > peakCumPct) peakCumPct = cumPct;
    const ddPoints = peakCumPct - cumPct;
    if (ddPoints > maxDdPctPoints) maxDdPctPoints = ddPoints;

    const tradeReturn = (pnl / 100) * stakeFraction;
    equity = Math.max(0, equity * (1 + tradeReturn));
    if (equity > peakEquity) peakEquity = equity;
    const ddEq = peakEquity > 0 ? ((peakEquity - equity) / peakEquity) * 100 : 0;
    if (ddEq > maxDdEquityPct) maxDdEquityPct = ddEq;
  }

  return {
    maxDdPctPoints: Number(maxDdPctPoints.toFixed(2)),
    maxDdEquityPct: Number(maxDdEquityPct.toFixed(2)),
    endingEquityIndex: Number(equity.toFixed(2)),
  };
}

// Estimate 2x slippage PnL when reading legacy paper_close records that lack pnl2xSlip.
export function estimateLegacy2xSlipPnl(row, entrySlip = cfg.entrySlip, exitSlip = cfg.exitSlip)
    {
  if (Number.isFinite(Number(row?.pnl2xSlip))) return Number(row.pnl2xSlip);
  if (Number.isFinite(Number(row?.rawEntryQuote)) && Number.isFinite(Number(row?.rawExitQuote)))
    {
    const exact = computeDoubleSlipPnl({
      rawEntryQuote: Number(row.rawEntryQuote),
      rawExitQuote: Number(row.rawExitQuote),
      entrySlip,
      exitSlip,
    });
    if (exact !== null) return exact;
  }
  const pnl = Number(row?.pnl);
  if (!Number.isFinite(pnl)) return null;
  // Conservative multiplicative adjustment from 1x (entrySlip, exitSlip) to 2x (2*entrySlip, 2*
    exitSlip)
  const grossRatio = (1 + pnl / 100) * ((1 + entrySlip) / Math.max(0.01, 1 - exitSlip));
  const netRatio2x = grossRatio * (Math.max(0.01, 1 - 2 * exitSlip) / (1 + 2 * entrySlip));
  return Number(((netRatio2x - 1) * 100).toFixed(2));
}

export function parseJsonlText(text) {
  const rows = [];
  let malformedLines = 0;
  for (const line of String(text).split("\n")) {
    const trimmed = line.trim();
    if (!trimmed) continue;
    try {
      rows.push(JSON.parse(trimmed));
    } catch {
      malformedLines += 1;
    }
  }
  return { rows, malformedLines };
}

export function analyzeLogs(rows, { malformedLines = 0, activePositions = [], archivedPositions
    = [] } = {}) {
  const gates = rows.filter((r) => r && r.ev === "gate");
  const gateErrors = rows.filter((r) => r && r.ev === "gate_error");
  const gaps = rows.filter((r) => r && r.ev === "signal_gap");
  const skips = rows.filter((r) => r && (r.ev === "signal_skip" || r.ev === "halted_skip" || Str
    ing(r.ev).startsWith("skip_")));
  const opened = rows.filter((r) => r && r.ev === "paper_open");
  const closed = rows.filter((r) => r && r.ev === "paper_close" && Number.isFinite(Number(r.pnl)
    ));
  const unresolvedEvents = rows.filter((r) => r && r.ev === "paper_unresolved");
  const recoveredEvents = rows.filter((r) => r && r.ev === "paper_quote_recovered");
```

```
const archivedEvents = rows.filter((r) => r && r.ev === "paper_archived_unresolved");

const lags = gates
  .map((g) => Number(g.lagMs ?? g.receivedAgeMs))
  .filter((x) => Number.isFinite(x) && x >= 0)
  .sort((a, b) => a - b);

const chronPnls = closed.map((r) => Number(r.pnl));
const sortedPnls = [...chronPnls].sort((a, b) => a - b);
const pnls2x = closed
  .map((r) => estimateLegacy2xSlipPnl(r))
  .filter((x) => Number.isFinite(x));

const byReason = {};
for (const r of closed) {
  const key = r.reason ?? "unknown";
  (byReason[key] ??= []).push(Number(r.pnl));
}

const bySkipReason = {};
for (const r of skips) {
  const key = r.reason ?? r.ev;
  bySkipReason[key] = (bySkipReason[key] ?? 0) + 1;
}

// Reconstruct unclosed positions from log history if activePositions not supplied
const openMap = new Map();
for (const r of rows) {
  if (!r || !r.mint) continue;
  if (r.ev === "paper_open") openMap.set(r.mint, r);
  if (r.ev === "paper_close" || r.ev === "paper_archived_unresolved") openMap.delete(r.mint);
}
const unresolvedActive = activePositions.length
  ? activePositions
  : [...openMap.values()];

const archivedList = archivedEvents.length ? archivedEvents : archivedPositions;
const archivedUnresolvedCount = archivedList.length;

// Separate worst-case stress estimate if archived unresolved positions lost 100% of their rem
  aining unclosed weight
const stressedArchPnls = archivedList.map((a) => {
  const banked = Number.isFinite(Number(a?.realizedBankedPct ?? a?.realized))
    ? Number(a.realizedBankedPct ?? a.realized)
    : 0;
  const remWeight = a?.tookHalf ? 0.5 : 1.0;
  return Number((banked + remWeight * -100).toFixed(2));
});
const combinedStressedPnls = [...chronPnls, ...stressedArchPnls];
const stressedMeanPnl = combinedStressedPnls.length
  ? Number(mean(combinedStressedPnls).toFixed(2))
  : null;
const stressedMedianPnl = combinedStressedPnls.length
  ? Number(median(combinedStressedPnls).toFixed(2))
  : null;

const totalCompletedOrArchived = closed.length + archivedUnresolvedCount;
const archivedSharePct =
  totalCompletedOrArchived > 0
    ? Number(((archivedUnresolvedCount / totalCompletedOrArchived) * 100).toFixed(1))
    : 0;
const archivedBiasWarning = archivedSharePct > 5;

const dd = computeDrawdown(chronPnls, 0.1);
const meanPnl = Number(mean(sortedPnls).toFixed(2));
const medianPnl = Number(median(sortedPnls).toFixed(2));
const meanPnl2x = Number(mean(pnls2x).toFixed(2));
const medianPnl2x = Number(median(pnls2x).toFixed(2));

const minTradesTarget = 200;
```

```
  // Archived positions are strictly excluded from the 200-closed-trade target
  const sampleAdequate = closed.length >= minTradesTarget;
  const passesEvalGate =
    sampleAdequate &&
    meanPnl > 0 &&
    medianPnl > 0 &&
    meanPnl2x > 0 &&
    medianPnl2x > 0;

  return {
    malformedLines,
    signalsGated: gates.length,
    signalsPassed: gates.filter((g) => g.pass).length,
    gateErrors: gateErrors.length,
    signalGaps: gaps.length,
    missingSequenceTotal: sum(gaps.map((g) => Number(g.missingSequenceCount ?? 0)).filter(Number
    .isFinite)),
    lagMedianMs: lags.length ? Number(median(lags).toFixed(1)) : null,
    lagP90Ms: lags.length ? Number(percentile(lags, 90).toFixed(1)) : null,
    openedCount: opened.length,
    closedCount: closed.length,
    unresolvedOpenCount: unresolvedActive.length,
    unresolvedMissingQuoteEvents: unresolvedEvents.length,
    quoteRecoveredEvents: recoveredEvents.length,
    archivedUnresolvedCount,
    archivedSharePct,
    archivedBiasWarning,
    stressedMeanPnl,
    stressedMedianPnl,
    winRatePct: sortedPnls.length
      ? Number(((sortedPnls.filter((x) => x > 0).length / sortedPnls.length) * 100).toFixed(1))
      : 0,
    meanPnl,
    medianPnl,
    bestPnl: sortedPnls.length ? sortedPnls.at(-1) : 0,
    worstPnl: sortedPnls.length ? sortedPnls[0] : 0,
    meanPnl2x,
    medianPnl2x,
    drawdown: dd,
    byReason,
    bySkipReason,
    evaluationGate: {
      minTradesTarget,
      sampleAdequate,
      passesEvalGate,
      archivedBiasWarning,
    },
  };
}

export function formatReport(summary) {
  const lines = [];
  lines.push(
    `Signals gated: ${summary.signalsGated}  passed: ${summary.signalsPassed}  gate_errors: ${su
    mmary.gateErrors}`,
  );
  if (summary.signalGaps > 0) {
    lines.push(`Queue gaps detected: ${summary.signalGaps} (missed seq IDs: ${summary.missingSeq
    uenceTotal})`);
  }
  if (summary.lagMedianMs !== null) {
    lines.push(
      `Relay lag ms (Worker receipt -> phone gate)  median: ${summary.lagMedianMs}  p90: ${summa
    ry.lagP90Ms}`,
    );
  }
  lines.push(
    `Positions opened: ${summary.openedCount}  Closed trades: ${summary.closedCount}  Unresolved
    /open: ${summary.unresolvedOpenCount}  Archived unresolved: ${summary.archivedUnresolvedCount}`,
  );
```

```
  if (summary.unresolvedMissingQuoteEvents > 0 || summary.quoteRecoveredEvents > 0 || summary.ar
    chivedUnresolvedCount > 0) {
    lines.push(
      `Missing-quote events -> unresolved: ${summary.unresolvedMissingQuoteEvents}  recovered: $
    {summary.quoteRecoveredEvents}  archived: ${summary.archivedUnresolvedCount} (${summary.archived
    SharePct}% of completed/archived)`,
    );
  }
  if (summary.archivedUnresolvedCount > 0 && summary.stressedMeanPnl !== null) {
    lines.push(
      `Worst-case unresolved stress estimate (if archived lost remaining weight): Mean ${summary
    .stressedMeanPnl.toFixed(2)}%  Median ${summary.stressedMedianPnl.toFixed(2)}%`,
    );
  }
  if (summary.archivedBiasWarning) {
    lines.push(
      `Bias warning: archived unresolved positions exceed 5% (${summary.archivedSharePct}%) of c
    ompleted/archived outcomes.`,
    );
  }
  if (summary.malformedLines > 0) {
    lines.push(`Warning: ignored ${summary.malformedLines} malformed JSONL line(s)`);
  }

  if (summary.closedCount > 0) {
    lines.push(`Win rate: ${summary.winRatePct.toFixed(1)}%`);
    lines.push(`Mean pnl: ${summary.meanPnl.toFixed(2)}%  Median: ${summary.medianPnl.toFixed(2)
    }%`);
    lines.push(
      `2x Slippage sensitivity -> Mean pnl: ${summary.meanPnl2x.toFixed(2)}%  Median: ${summary.
    medianPnl2x.toFixed(2)}%`,
    );
    lines.push(`Best: ${summary.bestPnl}%  Worst: ${summary.worstPnl}%`);
    lines.push(
      `Max drawdown: ${summary.drawdown.maxDdPctPoints.toFixed(2)} pct-pts (10%-stake equity DD:
     ${summary.drawdown.maxDdEquityPct.toFixed(2)}%, ending index: ${summary.drawdown.endingEquityIn
    dex})`,
    );
    lines.push("By exit reason:");
    for (const [k, v] of Object.entries(summary.byReason)) {
      lines.push(
        `  ${k.padEnd(22)} n=${v.length}  mean=${mean(v).toFixed(1)}%  median=${median(v).toFixe
    d(1)}%`,
      );
    }
  }

  const evalStatus = summary.evaluationGate.passesEvalGate
    ? "PASS (eligible for next-stage discussion; paper-only remains enforced)"
    : `INCOMPLETE/FAIL (closed=${summary.closedCount}/${summary.evaluationGate.minTradesTarget},
     mean2x=${summary.meanPnl2x.toFixed(2)}%, median2x=${summary.medianPnl2x.toFixed(2)}%)`;
  lines.push(`Evaluation gate (>=200 trades & positive 1x/2x mean+median): ${evalStatus}`);

  return lines.join("\n");
}

const isDirectRun =
  process.argv[1] && fileURLToPath(import.meta.url) === process.argv[1];

if (isDirectRun) {
  const file = process.argv[2] || "logs/paper.jsonl";
  if (!existsSync(file)) {
    console.log("No log yet.");
    process.exit(0);
  }
  const { rows, malformedLines } = parseJsonlText(readFileSync(file, "utf8"));
  let activePositions = [];
  let archivedPositions = [];
  if (existsSync("logs/positions.json")) {
    try {
```

```
      const parsed = JSON.parse(readFileSync("logs/positions.json", "utf8"));
      if (Array.isArray(parsed?.positions)) activePositions = parsed.positions;
      if (Array.isArray(parsed?.archivedUnresolved)) archivedPositions = parsed.archivedUnresolv
    ed;
    } catch {
      /* ignore corrupt positions file in report */
    }
  }
  const summary = analyzeLogs(rows, { malformedLines, activePositions, archivedPositions });
  console.log(formatReport(summary));
}
```

File Path: worker/index.js

```
// Cloudflare Worker relay (paste into the dashboard editor).
// Helius webhook -> KV queue -> phone polls /signals. Also: Telegram alerts, heartbeat watchdog
    , kill switch.
// Bindings needed: KV (KV namespace). Secrets: APP_KEY, HELIUS_AUTH, TG_TOKEN, TG_CHAT.
// Cron trigger: * * * * *
// Note: Cloudflare KV is eventually consistent; concurrent webhook requests can experience read
    -modify-write races.

const WSOL = "So11111111111111111111111111111111111111112";
const USDC = "EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v";
const USDT = "Es9vMFrzaCERmJfrF4H2FYD4KCoNkY11McCe8BenwNYB";
const IGNORE = new Set([WSOL, USDC, USDT]);
const QMAX = 50;                    // keep the last 50 signals
const HB_LATE_MS = 12 * 60 * 1000;  // alert if the phone is silent this long

const json = (o, status = 200) =>
  new Response(JSON.stringify(o), {
    status,
    headers: { "content-type": "application/json" },
  });

async function tg(env, text) {
  if (!env.TG_TOKEN || !env.TG_CHAT) return;
  await fetch(‘https://api.telegram.org/bot${env.TG_TOKEN}/sendMessage‘, {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify({
      chat_id: env.TG_CHAT,
      text: String(text).slice(0, 3500),
    }),
  });
}

// Adapt this after verifying actual Helius webhook payloads.
export function extractMints(tx) {
  const out = new Set();
  for (const t of tx?.tokenTransfers ?? []) {
    if (typeof t?.mint === "string" && t.mint && !IGNORE.has(t.mint)) {
      out.add(t.mint);
    }
  }
  return [...out];
}

const readQ = async (env) =>
  (await env.KV.get("queue", "json")) ?? {
    seq: 0,
    halt: false,
    items: [],
  };

export default {
  async fetch(req, env) {
    const url = new URL(req.url);
    const path = url.pathname;

    // Helius webhook (its own auth header).
```

```
  if (path === "/hook" && req.method === "POST") {
    if (!env.HELIUS_AUTH || req.headers.get("authorization") !== env.HELIUS_AUTH) {
      return json({ error: "unauthorized" }, 401);
    }

    const body = await req.json().catch(() => null);
    if (body === null || typeof body !== "object") {
      return json({ error: "invalid_json" }, 400);
    }

    const txs = Array.isArray(body) ? body : [body];
    const q = await readQ(env);
    let changed = false;

    for (const tx of txs) {
      if (!tx || typeof tx !== "object") continue;
      const receivedAt = Date.now();
      const sig = typeof tx.signature === "string" ? tx.signature.trim() : "";
      const chainTs = Number.isFinite(Number(tx.timestamp)) ? Number(tx.timestamp) : null;

      for (const mint of extractMints(tx)) {
        // Deduplicate a transaction/mint pair, not all future events for a mint.
        // Missing signatures cannot be reliably deduplicated across retries.
        const duplicate =
          sig && q.items.some((i) => i.sig === sig && i.mint === mint);
        if (duplicate) continue;

        q.seq += 1;
        q.items.push({
          id: q.seq,
          mint,
          sig,
          ts: receivedAt,
          ...(chainTs !== null ? { chainTs } : {}),
        });
        changed = true;
      }
    }

    if (changed) {
      q.items = q.items.slice(-QMAX);
      await env.KV.put("queue", JSON.stringify(q));
    }

    return json({ ok: true });
  }

  // Everything below requires the app key.
  if (!env.APP_KEY || req.headers.get("x-key") !== env.APP_KEY) {
    return json({ error: "unauthorized" }, 401);
  }

  if (path === "/signals" && req.method === "GET") {
    const rawSince = url.searchParams.get("since");
    const parsedSince = rawSince === null ? -1 : Number(rawSince);
    const since = Number.isFinite(parsedSince) ? parsedSince : -1;
    const q = await readQ(env);

    // since=0 requests retained backlog; the phone filters stale entries.
    const signals =
      since < 0 ? [] : q.items.filter((i) => i.id > since);
    const oldestId = q.items.length ? q.items[0].id : q.seq + 1;

    return json({
      signals,
      seq: q.seq,
      oldestId,
      halt: q.halt,
    });
  }
```

```
    if (path === "/heartbeat" && req.method === "POST") {
      const b = await req.json().catch(() => ({}));
      const open = Number.isFinite(Number(b?.open)) ? Number(b.open) : 0;
      await env.KV.put(
        "hb",
        JSON.stringify({ ts: Date.now(), open }),
      );
      return json({ ok: true });
    }

    if (path === "/notify" && req.method === "POST") {
      const b = await req.json().catch(() => ({}));
      await tg(env, b?.text ?? "");
      return json({ ok: true });
    }

    // Kill switch: POST /halt?on=1 to stop new entries, on=0 to resume.
    if (path === "/halt" && req.method === "POST") {
      const q = await readQ(env);
      q.halt = url.searchParams.get("on") === "1";
      await env.KV.put("queue", JSON.stringify(q));
      await tg(env, q.halt ? "HALT set: no new entries" : "HALT cleared");
      return json({ halt: q.halt });
    }

    return json({ error: "not found" }, 404);
  },

  // Dead-man's switch: runs every minute via cron.
  async scheduled(_event, env) {
    const hb = await env.KV.get("hb", "json");
    if (!hb) return;
    const late = Date.now() - hb.ts > HB_LATE_MS;
    const alerted = await env.KV.get("alerted");

    if (late && !alerted) {
      const mins = Math.round((Date.now() - hb.ts) / 60000);
      await tg(
        env,
        `Phone silent for ${mins} min. Open paper positions at last heartbeat: ${hb.open}`,
      );
      await env.KV.put("alerted", "1", { expirationTtl: 3600 });
    } else if (!late && alerted) {
      await env.KV.delete("alerted");
    }
  },
};
```

File Path: test/signals.test.js

```
import test from "node:test";
import assert from "node:assert/strict";
import { signalAgeMs, signalKey, sequenceGap } from "../src/signals.js";
import { loadConfig, parseNum } from "../src/config.js";
import { openPaper, openCount, isOpen, _resetPaperStateForTests } from "../src/paper.js";
import { handle, pollOnce, shutdown, _resetMainStateForTests } from "../src/main.js";
import worker from "../worker/index.js";

test("signal identity distinguishes transactions for the same mint", () => {
  assert.notEqual(
    signalKey({ id: 1, sig: "tx-a", mint: "mint-x" }),
    signalKey({ id: 2, sig: "tx-b", mint: "mint-x" }),
  );
});

test("signal identity deduplicates the same transaction and mint", () => {
  assert.equal(
    signalKey({ id: 1, sig: "tx-a", mint: "mint-x" }),
    signalKey({ id: 2, sig: "tx-a", mint: "mint-x" }),
  );
```

```
});

test("unsigned signals use queue identity instead of mint-only identity", () => {
  assert.notEqual(
    signalKey({ id: 1, mint: "mint-x" }),
    signalKey({ id: 2, mint: "mint-x" }),
  );
});

test("signal age is measurable and missing timestamps are explicit", () => {
  assert.equal(signalAgeMs({ ts: 1000 }, 2500), 1500);
  assert.equal(signalAgeMs({}, 2500), null);
});

test("queue gap counts IDs that could have fallen out of the bounded queue", () => {
  assert.equal(sequenceGap(10, 15), 4);
  assert.equal(sequenceGap(10, 11), 0);
  assert.equal(sequenceGap(-1, 15), 0);
});

test("config numeric validation rejects NaN and out-of-range values", () => {
  assert.equal(parseNum({ MAX_OPEN: "7" }, "MAX_OPEN", 5, { min: 1, max: 100 }), 7);
  assert.throws(() => parseNum({ MAX_OPEN: "abc" }, "MAX_OPEN", 5, { min: 1, max: 100 }), /Inval
    id numeric env var/);
  assert.throws(() => parseNum({ ENTRY_SLIP: "-0.1" }, "ENTRY_SLIP", 0.05, { min: 0, max: 0.95 }
    ), /Invalid numeric env var/);
  assert.throws(
    () => loadConfig({ RPC_URL: "", WORKER_URL: "https://w", APP_KEY: "k" }, { allowPlaceholder:
     false }),
    /Missing env var RPC_URL/,
  );
});

test("openPaper enforces MAX_OPEN atomically across concurrent entry attempts", async () => {
  _resetPaperStateForTests();
  const events = [];
  const logFn = (ev, data) => events.push({ ev, ...data });
  const notifyFn = () => {};
  const slowQuote = async (mint) => {
    await new Promise((r) => setTimeout(r, 20));
    return { price: 1.0, liqUsd: 25000, pair: `pair-${mint}` };
  };

  // Launch 4 concurrent openPaper calls with maxOpen = 2 and duplicate mint-1
  await Promise.all([
    openPaper("mint-1", "sig-1a", { signalId: 1 }, { getQuote: slowQuote, log: logFn, notify: no
    tifyFn, maxOpen: 2, minLiqUsd: 15000, entrySlip: 0.05, stateFile: null }),
    openPaper("mint-1", "sig-1b", { signalId: 2 }, { getQuote: slowQuote, log: logFn, notify: no
    tifyFn, maxOpen: 2, minLiqUsd: 15000, entrySlip: 0.05, stateFile: null }),
    openPaper("mint-2", "sig-2", { signalId: 3 }, { getQuote: slowQuote, log: logFn, notify: not
    ifyFn, maxOpen: 2, minLiqUsd: 15000, entrySlip: 0.05, stateFile: null }),
    openPaper("mint-3", "sig-3", { signalId: 4 }, { getQuote: slowQuote, log: logFn, notify: not
    ifyFn, maxOpen: 2, minLiqUsd: 15000, entrySlip: 0.05, stateFile: null }),
  ]);

  assert.equal(openCount(), 2);
  assert.equal(isOpen("mint-1"), true);
  assert.equal(isOpen("mint-2"), true);
  assert.equal(isOpen("mint-3"), false);

  const opened = events.filter((e) => e.ev === "paper_open");
  assert.equal(opened.length, 2);
  const dupSkips = events.filter((e) => e.ev === "signal_skip" && e.reason === "position_already
    _open");
  assert.equal(dupSkips.length, 1);
  const maxOpenSkips = events.filter((e) => e.ev === "skip_maxopen");
  assert.equal(maxOpenSkips.length, 1);
  _resetPaperStateForTests();
});
```

```
test("pollOnce processes signals sequentially, logs queue gaps, skips stale signals, and preserv
    es lagMs", async () => {
  _resetMainStateForTests(10);
  _resetPaperStateForTests();
  const events = [];
  const logFn = (ev, data) => events.push({ ev, ...data });
  const order = [];

  await pollOnce({
    getSignals: async (since) => {
      assert.equal(since, 10);
      return {
        seq: 16,
        oldestId: 14,
        halt: false,
        signals: [
          { id: 14, mint: "mint-stale", sig: "sig-14", ts: 1000 },
          { id: 15, mint: "mint-fresh-1", sig: "sig-15", ts: 95000 },
          { id: 16, mint: "mint-fresh-2", sig: "sig-16", ts: 96000 },
        ],
      };
    },
    runGate: async (mint) => {
      order.push(`gate:${mint}`);
      await new Promise((r) => setTimeout(r, 10));
      return { pass: true, reasons: [], top10Pct: 12.5 };
    },
    openPaper: async (mint, sig, meta) => {
      order.push(`open:${mint}`);
      logFn("paper_open", { mint, sig, ...meta });
    },
    isOpen: () => false,
    isHalted: () => false,
    log: logFn,
    maxSignalAgeMs: 60000,
    now: () => 100000,
  });

  // Verify gap of 3 signals (11, 12, 13) logged
  const gapEv = events.find((e) => e.ev === "signal_gap");
  assert.ok(gapEv);
  assert.equal(gapEv.missingSequenceCount, 3);

  // Verify stale signal skipped
  const staleEv = events.find((e) => e.ev === "signal_skip" && e.mint === "mint-stale");
  assert.ok(staleEv);
  assert.equal(staleEv.reason, "stale_signal");

  // Verify sequential execution order (gate 1 -> open 1 -> gate 2 -> open 2)
  assert.deepEqual(order, [
    "gate:mint-fresh-1",
    "open:mint-fresh-1",
    "gate:mint-fresh-2",
    "open:mint-fresh-2",
  ]);

  // Verify gate event includes lagMs for backward compatibility with report.js
  const gateEv = events.find((e) => e.ev === "gate" && e.mint === "mint-fresh-1");
  assert.ok(gateEv);
  assert.equal(gateEv.lagMs, 5000);
  assert.equal(gateEv.receivedAgeMs, 5000);
});

test("worker deduplicates by (sig, mint) while allowing new transactions for the same mint", asy
    nc () => {
  const store = new Map();
  const env = {
    HELIUS_AUTH: "secret-helius",
    APP_KEY: "secret-app",
    KV: {
```

```
      get: async (k, type) => {
        const v = store.get(k);
        if (!v) return null;
        return type === "json" ? JSON.parse(v) : v;
      },
      put: async (k, v) => {
        store.set(k, v);
      },
    },
  };

  const postHook = (body) =>
    worker.fetch(
      new Request("https://worker.local/hook", {
        method: "POST",
        headers: { authorization: "secret-helius", "content-type": "application/json" },
        body: JSON.stringify(body),
      }),
      env,
    );

  // First tx for mint-A
  let res = await postHook([{ signature: "sig-1", tokenTransfers: [{ mint: "mint-A" }] }]);
  assert.equal(res.status, 200);

  // Retry of same tx (sig-1, mint-A) -> deduplicated
  await postHook([{ signature: "sig-1", tokenTransfers: [{ mint: "mint-A" }] }]);

  // Second distinct tx (sig-2, mint-A) -> allowed!
  await postHook([{ signature: "sig-2", tokenTransfers: [{ mint: "mint-A" }] }]);

  const sigRes = await worker.fetch(
    new Request("https://worker.local/signals?since=0", {
      method: "GET",
      headers: { "x-key": "secret-app" },
    }),
    env,
  );
  assert.equal(sigRes.status, 200);
  const data = await sigRes.json();
  assert.equal(data.seq, 2);
  assert.equal(data.oldestId, 1);
  assert.equal(data.signals.length, 2);
  assert.equal(data.signals[0].sig, "sig-1");
  assert.equal(data.signals[1].sig, "sig-2");
});

test("shutdown handler is idempotent across SIGINT/SIGTERM/SIGHUP, persists state first, and doe
    s not exit in tests", () => {
  _resetMainStateForTests(0);
  const order = [];
  const events = [];

  const first = shutdown("SIGTERM", {
    savePositions: () => {
      order.push("savePositions");
      return true;
    },
    log: (ev, data) => {
      order.push(`log:${ev}`);
      events.push({ ev, ...data });
    },
    openCount: () => 3,
  });

  // Subsequent signals (SIGINT, SIGHUP) are no-ops
  const second = shutdown("SIGINT", {
    savePositions: () => {
      order.push("savePositions-duplicate");
      return true;
```

```
    },
    log: (ev) => order.push(`log-duplicate:${ev}`),
    openCount: () => 3,
  });
  const third = shutdown("SIGHUP", {
    savePositions: () => {
      order.push("savePositions-duplicate-2");
      return true;
    },
    log: (ev) => order.push(`log-duplicate-2:${ev}`),
    openCount: () => 3,
  });

  assert.equal(first, true);
  assert.equal(second, false);
  assert.equal(third, false);
  assert.deepEqual(order, ["savePositions", "log:stop"]);
  assert.equal(events.length, 1);
  assert.equal(events[0].signal, "SIGTERM");
  assert.equal(events[0].persisted, true);
  assert.equal(events[0].openPositions, 3);
  _resetMainStateForTests(0);
});
```

File Path: test/paper.test.js

```
import test from "node:test";
import assert from "node:assert/strict";
import { mkdtempSync, rmSync, writeFileSync } from "node:fs";
import { tmpdir } from "node:os";
import { join } from "node:path";
import { selectPair } from "../src/price.js";
import {
  openPaper,
  tick,
  evaluate,
  computeDoubleSlipPnl,
  loadPositions,
  openCount,
  isOpen,
  getOpenPositions,
  getArchivedPositions,
  _resetPaperStateForTests,
} from "../src/paper.js";

test("selectPair prefers entry pairAddress when still valid and flags fallback when switched", (
    ) => {
  const pairs = [
    { chainId: "solana", pairAddress: "pair-A", priceUsd: "1.00", liquidity: { usd: 20000 } },
    { chainId: "solana", pairAddress: "pair-B", priceUsd: "1.50", liquidity: { usd: 50000 } },
  ];

  // Initial entry selects highest liquidity (pair-B)
  const initial = selectPair(pairs);
  assert.equal(initial.pair, "pair-B");
  assert.equal(initial.price, 1.5);
  assert.equal(initial.pairSwitched, false);

  // Subsequent quote with preferredPair="pair-A" stays on pair-A even though pair-B has higher
    liquidity
  const sticky = selectPair(pairs, "pair-A");
  assert.equal(sticky.pair, "pair-A");
  assert.equal(sticky.price, 1.0);
  assert.equal(sticky.pairSwitched, false);

  // If preferredPair disappears, falls back to highest liquidity and sets pairSwitched=true
  const switched = selectPair(pairs, "pair-missing");
  assert.equal(switched.pair, "pair-B");
  assert.equal(switched.pairSwitched, true);
});
```

```
test("missing quotes do not fabricate a -100% total loss, reset missingSince on recovery, and ca
    n close cleanly", async () => {
  _resetPaperStateForTests();
  const events = [];
  const logFn = (ev, data) => events.push({ ev, ...data });
  const notifyFn = () => {};

  await openPaper(
    "mint-missing",
    "sig-m1",
    { signalId: 10 },
    {
      getQuote: async () => ({ price: 1.0, liqUsd: 25000, pair: "pair-1" }),
      log: logFn,
      notify: notifyFn,
      entrySlip: 0.05,
      stateFile: null,
      now: () => 1000,
    },
  );
  assert.equal(openCount(), 1);

  // Simulate 3 missing quotes with noQuoteTicks = 3 (not yet reaching maxUnresolvedMin = 30)
  for (let i = 1; i <= 3; i++) {
    await tick({
      getQuote: async () => null,
      log: logFn,
      notify: notifyFn,
      stateFile: null,
      noQuoteTicks: 3,
      maxUnresolvedMin: 30,
      now: () => 1000 + i * 5000,
    });
  }

  // Position MUST remain open (not closed at -100%) and emit paper_unresolved once
  assert.equal(openCount(), 1);
  assert.equal(events.some((e) => e.ev === "paper_close"), false);
  const unresolved = events.filter((e) => e.ev === "paper_unresolved");
  assert.equal(unresolved.length, 1);
  assert.equal(unresolved[0].misses, 3);
  assert.equal(unresolved[0].missingSince, 6000);
  assert.equal(unresolved[0].lastQuotePrice, 1.0);

  // Quote recovers at t=25000 (price still in band -> stays open and resets missingSince)
  await tick({
    getQuote: async () => ({ price: 1.1, liqUsd: 25000, pair: "pair-1" }),
    log: logFn,
    notify: notifyFn,
    stateFile: null,
    noQuoteTicks: 3,
    maxUnresolvedMin: 30,
    exitSlip: 0.08,
    now: () => 25000,
  });

  const openAfterRecovery = getOpenPositions()[0];
  assert.equal(openAfterRecovery.misses, 0);
  assert.equal(openAfterRecovery.missingSince, null);
  assert.equal(openAfterRecovery.unresolvedMissingQuote, false);

  const recovered = events.find((e) => e.ev === "paper_quote_recovered");
  assert.ok(recovered);
  assert.equal(recovered.missesBeforeRecovery, 3);
  assert.equal(recovered.missingDurationSec, 19);

  // Next tick triggers stop-loss cleanly with real quote
  await tick({
    getQuote: async () => ({ price: 0.5, liqUsd: 25000, pair: "pair-1" }),
    log: logFn,
```

```
    notify: notifyFn,
    stateFile: null,
    noQuoteTicks: 3,
    exitSlip: 0.08,
    now: () => 30000,
  });

  assert.equal(openCount(), 0);
  const closed = events.find((e) => e.ev === "paper_close");
  assert.ok(closed);
  assert.equal(closed.reason, "stop_loss");
  assert.notEqual(closed.pnl, -100);
  _resetPaperStateForTests();
});

test("long-unresolved positions archive safely after maxUnresolvedMin, never resurrect on restar
    t, and keep slot on disk failure", async () => {
  _resetPaperStateForTests();
  const dir = mkdtempSync(join(tmpdir(), "sniper-archive-test-"));
  const stateFile = join(dir, "positions.json");
  const events = [];
  const logFn = (ev, data) => events.push({ ev, ...data });
  const notifyFn = () => {};

  try {
    // Open position at t=1000 with maxOpen=1
    await openPaper(
      "mint-zombie",
      "sig-z1",
      { signalId: 50 },
      {
        getQuote: async () => ({ price: 1.0, liqUsd: 30000, pair: "pair-z" }),
        log: logFn,
        notify: notifyFn,
        maxOpen: 1,
        entrySlip: 0.05,
        stateFile,
        now: () => 1000,
      },
    );

    // Take partial profit (+100%) at t=5000 so realizedBankedPct > 0
    await tick({
      getQuote: async () => ({ price: 2.5, liqUsd: 40000, pair: "pair-z" }),
      log: logFn,
      notify: notifyFn,
      stateFile,
      exitSlip: 0.08,
      now: () => 5000,
    });
    assert.equal(getOpenPositions()[0].tookHalf, true);

    // Miss quotes starting at t=10_000 (misses=1, missingSince=10_000) and reach noQuoteTicks=2
     at t=15_000
    await tick({
      getQuote: async () => null,
      log: logFn,
      notify: notifyFn,
      stateFile,
      noQuoteTicks: 2,
      maxUnresolvedMin: 30,
      now: () => 10000,
    });
    await tick({
      getQuote: async () => null,
      log: logFn,
      notify: notifyFn,
      stateFile,
      noQuoteTicks: 2,
      maxUnresolvedMin: 30,
```

```
      now: () => 15000,
    });
    assert.equal(openCount(), 1);
    assert.equal(events.some((e) => e.ev === "paper_unresolved"), true);

    // At t = 10_000 + 30*60_000 = 1_810_000, simulate disk persistence failure first:
    // Slot MUST NOT be freed and paper_archived_unresolved MUST NOT be emitted if savePositions
     fails
    await tick({
      getQuote: async () => null,
      log: logFn,
      notify: notifyFn,
      stateFile,
      noQuoteTicks: 2,
      maxUnresolvedMin: 30,
      now: () => 1810000,
      writeFileSync: () => {
        throw new Error("simulated disk full");
      },
    });
    assert.equal(openCount(), 1);
    assert.equal(events.some((e) => e.ev === "paper_archived_unresolved"), false);

    // Now allow disk write to succeed at t = 1_815_000:
    await tick({
      getQuote: async () => null,
      log: logFn,
      notify: notifyFn,
      stateFile,
      noQuoteTicks: 2,
      maxUnresolvedMin: 30,
      now: () => 1815000,
    });

    // Active slot is now freed, paper_archived_unresolved is emitted, and paper_close is NEVER
    emitted
    assert.equal(openCount(), 0);
    assert.equal(events.some((e) => e.ev === "paper_close"), false);
    const archEv = events.find((e) => e.ev === "paper_archived_unresolved");
    assert.ok(archEv);
    assert.equal(archEv.mint, "mint-zombie");
    assert.equal(archEv.tookHalf, true);
    assert.ok(archEv.realizedBankedPct > 50);
    assert.equal(archEv.missingDurationSec, 1805);

    const archivedInMemory = getArchivedPositions();
    assert.equal(archivedInMemory.length, 1);
    assert.equal(archivedInMemory[0].mint, "mint-zombie");

    // Simulate restart and verify archived position is NOT resurrected into active positions
    _resetPaperStateForTests();
    const restored = loadPositions(stateFile, { log: logFn, maxOpen: 5 });
    assert.equal(restored, 0);
    assert.equal(openCount(), 0);
    assert.equal(getArchivedPositions().length, 1);
  } finally {
    rmSync(dir, { recursive: true, force: true });
    _resetPaperStateForTests();
  }
});

test("openPaper retries missing entry quotes up to entryQuoteRetries and aborts immediately if H
    ALT activates", async () => {
  _resetPaperStateForTests();
  const events = [];
  const logFn = (ev, data) => events.push({ ev, ...data });
  const notifyFn = () => {};

  // 1) Succeeds on 2nd attempt after 1 initial null quote
  let calls = 0;
```

```
  await openPaper(
    "mint-retry-ok",
    "sig-r1",
    { signalId: 88 },
    {
      getQuote: async () => {
        calls += 1;
        if (calls === 1) return null;
        return { price: 1.0, liqUsd: 25000, pair: "pair-r1" };
      },
      sleep: async () => {},
      isHalted: () => false,
      log: logFn,
      notify: notifyFn,
      entryQuoteRetries: 2,
      entryQuoteRetryMs: 10,
      stateFile: null,
    },
  );
  assert.equal(calls, 2);
  assert.equal(openCount(), 1);
  const openEv = events.find((e) => e.ev === "paper_open" && e.mint === "mint-retry-ok");
  assert.ok(openEv);
  assert.equal(openEv.quoteAttempts, 2);

  // 2) HALT activates during retry wait -> aborts with halted_skip and releases opening lock
  let halted = false;
  await openPaper(
    "mint-retry-halt",
    "sig-r2",
    { signalId: 89 },
    {
      getQuote: async () => null,
      sleep: async () => {
        halted = true;
      },
      isHalted: () => halted,
      log: logFn,
      notify: notifyFn,
      entryQuoteRetries: 2,
      entryQuoteRetryMs: 10,
      stateFile: null,
    },
  );
  assert.equal(isOpen("mint-retry-halt"), false);
  const haltSkip = events.find((e) => e.ev === "halted_skip" && e.mint === "mint-retry-halt");
  assert.ok(haltSkip);
  assert.equal(haltSkip.reason, "halt_after_quote_retry_wait");
  _resetPaperStateForTests();
});

test("partial take-profit + trailing stop and doubled-slippage PnL calculation are accurate", as
    ync () => {
  _resetPaperStateForTests();
  const events = [];
  const logFn = (ev, data) => events.push({ ev, ...data });
  const notifyFn = () => {};

  // Open at raw quote 1.00 -> entry = 1.05 (with 5% entrySlip)
  await openPaper(
    "mint-tp",
    "sig-tp",
    { signalId: 20 },
    {
      getQuote: async () => ({ price: 1.0, liqUsd: 30000, pair: "pair-tp" }),
      log: logFn,
      notify: notifyFn,
      entrySlip: 0.05,
      stateFile: null,
      now: () => 1000,
```

```
    },
  );

  // Price rises to 2.50 -> exec px = 2.50 * 0.92 = 2.30 -> +119.05% (> +100% takeProfit)
  // Takes half and arms trailing stop (peak = 2.30)
  await tick({
    getQuote: async () => ({ price: 2.5, liqUsd: 40000, pair: "pair-tp" }),
    log: logFn,
    notify: notifyFn,
    stateFile: null,
    exitSlip: 0.08,
    now: () => 6000,
  });

  assert.equal(openCount(), 1);
  const partial = events.find((e) => e.ev === "paper_partial");
  assert.ok(partial);
  assert.equal(partial.reason, "take_profit_half");

  // Price pulls back to 1.90 -> exec px = 1.90 * 0.92 = 1.748 (<= 2.30 * 0.80 = 1.84) -> traili
    ng_stop exit
  await tick({
    getQuote: async () => ({ price: 1.9, liqUsd: 35000, pair: "pair-tp" }),
    log: logFn,
    notify: notifyFn,
    stateFile: null,
    exitSlip: 0.08,
    now: () => 12000,
  });

  assert.equal(openCount(), 0);
  const closed = events.find((e) => e.ev === "paper_close");
  assert.ok(closed);
  assert.equal(closed.reason, "trailing_stop");
  assert.ok(closed.pnl > 80);
  assert.ok(Number.isFinite(closed.pnl2xSlip));
  assert.ok(closed.pnl2xSlip < closed.pnl);
  _resetPaperStateForTests();
});

test("positions persist to disk and recover safely on restart without duplication", async () =>
    {
  _resetPaperStateForTests();
  const dir = mkdtempSync(join(tmpdir(), "sniper-paper-test-"));
  const stateFile = join(dir, "positions.json");
  const events = [];
  const logFn = (ev, data) => events.push({ ev, ...data });

  try {
    await openPaper(
      "mint-persist",
      "sig-p1",
      { signalId: 77 },
      {
        getQuote: async () => ({ price: 2.0, liqUsd: 50000, pair: "pair-p1" }),
        log: logFn,
        notify: () => {},
        entrySlip: 0.05,
        stateFile,
        now: () => 5000,
      },
    );
    assert.equal(openCount(), 1);

    // Simulate process restart by clearing in-memory state and loading from disk
    _resetPaperStateForTests();
    assert.equal(openCount(), 0);

    const restoredCount = loadPositions(stateFile, { log: logFn, maxOpen: 5 });
    assert.equal(restoredCount, 1);
```

```
    assert.equal(openCount(), 1);
    assert.equal(isOpen("mint-persist"), true);

    const openList = getOpenPositions();
    assert.equal(openList[0].mint, "mint-persist");
    assert.equal(openList[0].pair, "pair-p1");
    assert.equal(openList[0].entry, 2.1);

    // Verify corrupt state file fails closed without crashing
    _resetPaperStateForTests();
    writeFileSync(stateFile, "{corrupt-json", "utf8");
    const badRestore = loadPositions(stateFile, { log: logFn, maxOpen: 5 });
    assert.equal(badRestore, 0);
    assert.equal(openCount(), 0);
  } finally {
    rmSync(dir, { recursive: true, force: true });
    _resetPaperStateForTests();
  }
});
```

File Path: test/gate.test.js

```
import test from "node:test";
import assert from "node:assert/strict";
import { runGate } from "../src/gate.js";
import { rpc, redactSecrets } from "../src/rpc.js";

const SYSTEM_PROGRAM = "11111111111111111111111111111111";
const RAYDIUM_PROGRAM = "675kPX9MHTjS2zt1qfr1NYHuzeLXfQM9H24wFSUt1Mp8";

test("runGate rejects tokens when mintAuthority or freezeAuthority is still active", async () =>
     {
  const res = await runGate(
    "mint-active-auth",
    { maxTop10Pct: 35 },
    {
      rpc: async (method) => {
        if (method === "getAccountInfo") {
          return {
            value: {
              data: {
                parsed: {
                  info: {
                    decimals: 6,
                    mintAuthority: "SomeAuthorityPubkey",
                    freezeAuthority: "SomeFreezePubkey",
                  },
                },
              },
            },
          };
        }
        if (method === "getTokenSupply") return { value: { uiAmount: 1000 } };
        if (method === "getTokenLargestAccounts") return { value: [] };
        return { value: [] };
      },
    },
  );

  assert.equal(res.pass, false);
  assert.ok(res.reasons.includes("mint authority not revoked"));
  assert.ok(res.reasons.includes("freeze authority not revoked"));
});

test("runGate aggregates multiple token accounts owned by the same wallet and excludes PDA vault
    s", async () => {
  // Supply = 1000.
  // Raydium PDA holds 500 (50% - should be excluded).
  // Wallet-A holds 3 token accounts of 80 each = 240 (24%).
  // Unfunded Wallet-B (null accountInfo) holds 150 (15%).
  // Combined wallet concentration = 39% (> 35% threshold).
```

```
  let callIndex = 0;
  const res = await runGate(
    "mint-sybil",
    { maxTop10Pct: 35 },
    {
      rpc: async (method) => {
        if (method === "getAccountInfo") {
          return {
            value: {
              data: {
                parsed: {
                  info: { decimals: 6, mintAuthority: null, freezeAuthority: null },
                },
              },
            },
          };
        }
        if (method === "getTokenSupply") {
          return { value: { uiAmount: 1000 } };
        }
        if (method === "getTokenLargestAccounts") {
          return {
            value: [
              { address: "ta-pda", uiAmount: 500 },
              { address: "ta-a1", uiAmount: 80 },
              { address: "ta-a2", uiAmount: 80 },
              { address: "ta-a3", uiAmount: 80 },
              { address: "ta-b1", uiAmount: 150 },
            ],
          };
        }
        if (method === "getMultipleAccounts") {
          callIndex += 1;
          if (callIndex === 1) {
            // Token account owners
            return {
              value: [
                { data: { parsed: { info: { owner: "owner-pda" } } } },
                { data: { parsed: { info: { owner: "wallet-A" } } } },
                { data: { parsed: { info: { owner: "wallet-A" } } } },
                { data: { parsed: { info: { owner: "wallet-A" } } } },
                { data: { parsed: { info: { owner: "wallet-B-unfunded" } } } },
              ],
            };
          }
          // Owner account info: owner-pda is owned by Raydium program, wallet-A by SYSTEM_PROGR
    AM, wallet-B-unfunded is null
          return {
            value: [
              { owner: RAYDIUM_PROGRAM },
              { owner: SYSTEM_PROGRAM },
              null,
            ],
          };
        }
        return { value: null };
      },
    },
  );

  assert.equal(res.top10Pct, 39);
  assert.equal(res.pass, false);
  assert.ok(res.reasons[0].includes("39.0%"));
});

test("rpc checks HTTP status (429/500), handles malformed JSON, and redacts API keys on network/
    RPC errors", async () => {
  const secretRpcUrl = "https://mainnet.helius-rpc.com/?api-key=SUPER_SECRET_HELIUS_KEY_999";

  // 1) HTTP 429 status check before JSON parsing
```

```
await assert.rejects(
  () =>
    rpc("getAccountInfo", ["mint-1"], {
      rpcUrl: secretRpcUrl,
      fetch: async () => ({
        ok: false,
        status: 429,
        json: async () => {
          throw new Error("should not parse JSON on HTTP 429");
        },
      }),
    }),
  (err) => {
    assert.match(err.message, /getAccountInfo: HTTP 429/);
    assert.equal(err.message.includes("SUPER_SECRET_HELIUS_KEY_999"), false);
    return true;
  },
);

// 2) HTTP 500 status check
await assert.rejects(
  () =>
    rpc("getTokenSupply", ["mint-1"], {
      rpcUrl: secretRpcUrl,
      fetch: async () => ({ ok: false, status: 500 }),
    }),
  /getTokenSupply: HTTP 500/,
);

// 3) Malformed JSON on HTTP 200
await assert.rejects(
  () =>
    rpc("getTokenSupply", ["mint-1"], {
      rpcUrl: secretRpcUrl,
      fetch: async () => ({
        ok: true,
        status: 200,
        json: async () => {
          throw new SyntaxError("Unexpected token < in JSON");
        },
      }),
    }),
  /getTokenSupply: malformed JSON response/,
);

// 4) Network failure with URL & API key in error message + cause
await assert.rejects(
  () =>
    rpc("getMultipleAccounts", [[]], {
      rpcUrl: secretRpcUrl,
      fetch: async () => {
        const e = new TypeError(`fetch failed for ${secretRpcUrl}`);
        e.cause = new Error("connect ECONNREFUSED ?api-key=SUPER_SECRET_HELIUS_KEY_999");
        throw e;
      },
    }),
  (err) => {
    assert.match(err.message, /getMultipleAccounts: network error:/);
    assert.equal(err.message.includes("SUPER_SECRET_HELIUS_KEY_999"), false);
    assert.ok(err.message.includes("[REDACTED]"));
    return true;
  },
);

// 5) JSON-RPC error object containing secret or authorization header
const redacted = redactSecrets(
  "Upstream failed at https://mainnet.helius-rpc.com/?api-key=MY_KEY_12345 with authorization:
   Bearer SECRET_BEARER_TOKEN",
  [secretRpcUrl],
);
```

```
  assert.equal(redacted.includes("MY_KEY_12345"), false);
  assert.equal(redacted.includes("SECRET_BEARER_TOKEN"), false);
});
```

File Path: test/report.test.js

```
import test from "node:test";
import assert from "node:assert/strict";
import {
  median,
  computeDrawdown,
  parseJsonlText,
  analyzeLogs,
  formatReport,
} from "../src/report.js";

test("median calculates exact middle for odd and average of two middles for even arrays", () =>
    {
  assert.equal(median([]), 0);
  assert.equal(median([10]), 10);
  assert.equal(median([10, 20]), 15);
  assert.equal(median([40, 10, 20, 30]), 25);
  assert.equal(median([10, 20, 90]), 20);
});

test("computeDrawdown tracks both cumulative pct-point drawdown and compounded 10%-stake equity
    drawdown", () => {
  const dd = computeDrawdown([50, -25, -25, 100], 0.1);
  // Cumulative pct: +50 -> +25 -> 0 (drawdown of 50 pct-pts from peak 50) -> +100
  assert.equal(dd.maxDdPctPoints, 50);
  assert.ok(dd.maxDdEquityPct > 4.5 && dd.maxDdEquityPct < 5.5);
  assert.ok(dd.endingEquityIndex > 105);
});

test("analyzeLogs is backward-compatible with legacy JSONL records, tracks archived unresolved s
    eparately, and handles malformed lines safely", () => {
  const rawJsonl = [
    JSON.stringify({ ts: "2026-10-28T00:00:01Z", ev: "gate", mint: "m1", pass: true, lagMs: 1200
     }),
    JSON.stringify({ ts: "2026-10-28T00:00:02Z", ev: "gate", mint: "m2", pass: false, receivedAg
    eMs: 2400 }),
    "{broken-json-line",
    JSON.stringify({ ts: "2026-10-28T00:00:03Z", ev: "paper_open", mint: "m1", entry: 1.05, liqU
    sd: 20000 }),
    JSON.stringify({ ts: "2026-10-28T00:00:04Z", ev: "paper_open", mint: "m3", entry: 2.1, liqUs
    d: 30000 }),
    JSON.stringify({ ts: "2026-10-28T00:00:05Z", ev: "paper_open", mint: "m5", entry: 1.0, liqUs
    d: 25000 }),
    JSON.stringify({ ts: "2026-10-28T00:00:10Z", ev: "paper_unresolved", mint: "m3", misses: 12
    }),
    JSON.stringify({ ts: "2026-10-28T00:00:11Z", ev: "paper_unresolved", mint: "m5", misses: 12
    }),
    JSON.stringify({ ts: "2026-10-28T00:35:00Z", ev: "paper_archived_unresolved", mint: "m5", to
    okHalf: true, realizedBankedPct: 50, missingDurationSec: 1800 }),
    // Legacy paper_close without pnl2xSlip
    JSON.stringify({ ts: "2026-10-28T00:00:20Z", ev: "paper_close", mint: "m1", reason: "trailin
    g_stop", pnl: 40, heldSec: 17 }),
    // New paper_close with explicit pnl2xSlip
    JSON.stringify({ ts: "2026-10-28T00:00:30Z", ev: "paper_close", mint: "m4", reason: "stop_lo
    ss", pnl: -25, pnl2xSlip: -34.8, heldSec: 12 }),
  ].join("\n");

  const { rows, malformedLines } = parseJsonlText(rawJsonl);
  assert.equal(malformedLines, 1);

  const summary = analyzeLogs(rows, { malformedLines });
  assert.equal(summary.signalsGated, 2);
  assert.equal(summary.signalsPassed, 1);
  assert.equal(summary.lagMedianMs, 1800); // average of 1200 and 2400!
  assert.equal(summary.closedCount, 2); // archived position m5 is NOT counted as a closed trade
  assert.equal(summary.unresolvedOpenCount, 1); // m3 remains open; m5 was archived
```

```
  assert.equal(summary.archivedUnresolvedCount, 1);
  assert.equal(summary.archivedBiasWarning, true); // 1 / (2 + 1) = 33.3% > 5%
  assert.equal(summary.meanPnl, 7.5); // primary mean strictly reflects closed trades (40 + -25)
     / 2
  assert.equal(summary.medianPnl, 7.5); // even-sized median of [40, -25]
  // Stressed PnL for m5 (tookHalf=true, realizedBankedPct=50 -> 50 + 0.5*-100 = 0), so [40, -25
    , 0] -> mean = 5.0
  assert.equal(summary.stressedMeanPnl, 5);
  assert.equal(summary.stressedMedianPnl, 0);
  assert.ok(summary.meanPnl2x < summary.meanPnl);
  assert.equal(summary.evaluationGate.sampleAdequate, false);
  assert.equal(summary.evaluationGate.passesEvalGate, false);

  const formatted = formatReport(summary);
  assert.ok(formatted.includes("2x Slippage sensitivity"));
  assert.ok(formatted.includes("Worst-case unresolved stress estimate"));
  assert.ok(formatted.includes("Bias warning:"));
  assert.ok(formatted.includes("Evaluation gate (>=200 trades"));
});
```